Kneo Agent Platform

Operation Guide

Version 1.0.0

2026-06-20

Deployment shapes, TLS termination, security hardening, observability, backup and recovery, upgrade, incident response, troubleshooting, and the deployment smoke test — one printable operator guide.

Table of contents

Deployment

Source: *docs/user/deployment.md*

Reference for the supported deployment shapes. For a guided zero-to-running walkthrough on Docker Compose with PostgreSQL, see `tutorial_postgres_deployment.md`. For copy-paste quick shapes (local poke, operator pull-and-run, one-shot CLI in the image), see `run_recipes.md`. For every environment variable referenced below, see `environment.md`.

The service supports three shapes:

- **Container** — a single `kneo-serv` image with a database you supply.
- **Compose** — the bundled stack that starts the API plus PostgreSQL.
- **Embedded** — `kneo_serv.service.app:create_app()` mounted in your own ASGI server (covered in `tutorial_custom_tool.md § 7`).

Container

Pull the published image from GitHub Container Registry:

```
docker pull ghcr.io/kneo-agent/kneo-serv:latest
```

Tag conventions: `<version>` (e.g. `0.9.0`), `<major>.<minor>` (`0.9`), and `latest`. amd64-only; arm64 is deferred to 2.0. From 0.3.0 onward the image is keyless-signed via cosign and ships with a CycloneDX SBOM attestation; from 0.4.0 onward the release pipeline also runs a blocking Trivy CVE scan against the pushed digest under the CVSS≥7 policy (security_hardening.md § Image vulnerability scanning) — verification commands in `supply_chain_review.md § Verification commands`. The image installs the `kneo-serv[deploy]` extra (psycopg + SDK telemetry).

Run it against PostgreSQL:

```
docker run --rm -p 8000:8000 \
  -e KNEO_SERV_DATABASE_URL=postgresql://kneo_serv:change-
me@host.docker.internal:5432/kneo_serv \
  -e KNEO_SERV_AUTH_ENABLED=true \
  -e KNEO_SERV_API_KEYS='operator:replace-token:operator' \
  ghcr.io/kneo-agent/kneo-serv:latest
```

For local builds from a source checkout (contributor / pre-publish):

```
docker build -t kneo-serv:local .
```

Compose

The bundled Compose stack starts the API plus PostgreSQL:

```
cp deploy/production.env.example deploy/production.env
docker compose --env-file deploy/production.env up --build
```

Replace every placeholder token and database password before binding the service to a network. The stack defaults to port `8000`; set `KNEO_SERV_PORT` to change the host-side port.

For a staging rehearsal, use the staging env example:

```
cp deploy/staging.env.example deploy/staging.env
KNEO_SERV_ENV_FILE=./deploy/staging.env \
  docker compose --env-file deploy/staging.env up --build
```

`deploy/staging.env` is gitignored. Keep SDK telemetry argument/result capture (`KNEO_SERV_OTEL_RECORD_ARGUMENTS`, `KNEO_SERV_OTEL_RECORD_RESULTS`) disabled in staging unless the deployment's data classification has explicitly approved payload capture.

TLS and reverse proxy

The service speaks bare HTTP and does not terminate TLS itself. For any deployment exposed beyond `127.0.0.1`, place a reverse proxy (nginx, Caddy, AWS ALB, or similar) in front and terminate TLS there. See `tls_and_proxy.md` for topology, bind-address guidance, and trusted-proxy header handling.

Choosing a persistence backend

Backend	When to use
SQLite	Local dev or single-process service. Default when `KNEO_SERV_DATABASE_URL` is unset.
PostgreSQL	Any multi-process or production deployment. Set `KNEO_SERV_DATABASE_URL`. Requires `kneo-serv[postgres]` or `kneo-serv[deploy]`.

When `KNEO_SERV_DATABASE_URL` is set, the service uses PostgreSQL for run state, checkpoints, idempotency records, queue leases, locks, audit events, and workflow continuations. Without it, the service falls back to SQLite for state and file-backed continuations.

Multi-process SQLite is not a supported topology; see troubleshooting.md § 2.3.

For the throughput and latency trade-offs between the two backends — including why SQLite write throughput does not scale with concurrency — and a bench harness to size your own deployment, see performance.md.

Readiness and liveness

Wire these endpoints into your supervisor or load balancer:

```
GET /livez      # process liveness
GET /readyz     # readiness: all dependencies healthy
GET /healthz    # lightweight overall health
```

`/livez` and `/readyz` are intentionally unauthenticated for probe integration. `/readyz` returns `503` with a structured `not_ready` payload when any dependency check fails — see troubleshooting.md § 1.2 for the failure shape.

The Prometheus scrape endpoint `GET /metrics` (since 0.5.0) is also unauthenticated — restrict it to your monitoring network or disable it with `KNEO_SERV_METRICS_ENABLED=false`. See observability.md § Prometheus `/metrics`.

Workers, scaling, and graceful shutdown

A single process runs a pool of `KNEO_SERV_WORKER_CONCURRENCY` worker threads (default `1`) draining the run queue. Raise it for provider-bound workloads; for write-concurrent scale on PostgreSQL, run **multiple service processes** — each leases queued runs safely via `FOR UPDATE SKIP LOCKED`. A starting CPU/RAM/worker floor to deploy against (and the SQLite single-writer caveat) is in performance.md § Minimum sizing.

On `SIGTERM` (e.g. a rolling deploy) the service drains the worker pool: workers stop claiming new work and finish the run they are currently executing, and the service waits up to `KNEO_SERV_SHUTDOWN_TIMEOUT_SECONDS` (default `30`) for them to exit. A run *still executing* when that timeout elapses is interrupted by process exit — but it stays claimed and is automatically re-leased and retried by another worker once its `KNEO_SERV_WORKER_LEASE_SECONDS` lease expires, so it is not lost, only restarted. To drain in-flight runs without that restart, set `KNEO_SERV_SHUTDOWN_TIMEOUT_SECONDS` (and your orchestrator's termination grace period) at least as long as your longest expected run step. Set `KNEO_SERV_MAX_QUEUE_DEPTH` to shed load with `503` under overload, and `KNEO_SERV_QUEUE_MAX_ATTEMPTS` (default `5`) to dead-letter poison runs rather than retry them forever.

Run with Docker

Source: *docs/user/run_recipes.md*

Three working ways to run the published `kneo-serv` Docker image. Pick the shape that matches what you're doing: quick local poke, real deployment, or one-shot CLI invocations.

For the reference of every supported deployment shape (including embedded ASGI), see `deployment.md`. For the guided zero-to-running PostgreSQL walkthrough, see `tutorial_postgres_deployment.md`.

All examples use `ghcr.io/kneo-agent/kneo-serv:latest`. Pin to a specific tag (`:<version>` or `:<major>.<minor>`) when you need reproducibility.

1. Quick kick-the-tires — single container, no auth, SQLite

The fastest way to see the service respond. No PostgreSQL, no API keys, no Compose — just one container with the SQLite-backed run state in an ephemeral volume.

```
docker run --rm -p 8000:8000 \
  -e KNEO_SERV_AUTH_ENABLED=false \
  ghcr.io/kneo-agent/kneo-serv:latest
```

In another terminal:

```
curl http://127.0.0.1:8000/healthz
# {"ok":true,"service":"kneo-serv-platform","version":"...",...}

curl http://127.0.0.1:8000/readyz
# {"ok":true,...,"checks":{"run_state_store":{"name":"run_state_store","ok":true},...}}
```

Run state lives at `/app/.kneo/kneo_runs.sqlite` inside the container and disappears when `--rm` cleans the container up. Don't use this shape for anything you want to keep around.

2. Production-ish — Compose stack with PostgreSQL sidecar

The operator-recommended path. Clone the repo to get `compose.yaml` and `deploy/production.env.example`:

```
git clone git@github.com:kneo-agent/kneo-serv.git
cd kneo-serv
```

```
cp deploy/production.env.example deploy/production.env
# Edit deploy/production.env — replace the `replace-*-token` API keys,
# set POSTGRES_PASSWORD, point providers at real keys, etc.

docker compose --env-file deploy/production.env pull
docker compose --env-file deploy/production.env up -d
```

`compose.yaml` defaults its `image:` to `ghcr.io/kneo-agent/kneo-serv:latest`. The `build:` block stays in place so contributors can run a local build with `docker compose up --build`. Auth is enabled by default in `production.env.example`; PostgreSQL provides durable run state and continuations.

Tear it down with:

```
docker compose --env-file deploy/production.env down -v
```

For the full walkthrough including the first authenticated request and how to migrate from a source build, see `tutorial_postgres_deployment.md`.

3. One-shot CLI usage inside the image

Spec validate / compile / one-shot run without keeping a service up. Useful for CI lanes, ad-hoc validation, and pre-flight checks against an artifact you don't want to install locally.

```
 # Validate a local spec by mounting it
docker run --rm -v "$PWD:/work" --entrypoint kneo \
  ghcr.io/kneo-agent/kneo-serv:latest \
  spec validate /work/my_spec.yaml

# CLI help
docker run --rm --entrypoint kneo \
  ghcr.io/kneo-agent/kneo-serv:latest --help
```

`--entrypoint kneo` is required because the `Dockerfile` only sets `CMD ["kneo", "service", "serve", ...]`, not `ENTRYPOINT`. Without the override, `docker run` treats the trailing args (`spec validate ...`) as the binary name and fails with `exec: "spec": executable file not found in $PATH`.

For the full CLI surface — local-state path, profile-backed service clients, run inspection, human-task resume — see `cli.md` and the generated `cli_reference.md`.

TLS and reverse proxy

Source: *docs/user/tls_and_proxy.md*

The Kneo Agent Platform service speaks plain HTTP. It does not terminate TLS, parse `X-Forwarded-*` headers itself, or rate-limit by IP. Any deployment that faces a network beyond `127.0.0.1` must run behind a reverse proxy that terminates TLS and shields the service.

For deployment shapes (Container, Compose, Embedded) and the choice of persistence backend, see `deployment.md`. For the hardening checklist that includes TLS, see `security_hardening.md`.

Topology

```
client ──HTTPS──► reverse proxy ──HTTP──► kneo-serv
                  (TLS termination,
                   request size limits,
                   rate limiting,
                   client-IP injection)
```

The proxy is responsible for:

- TLS termination and certificate management
- Request body size limits at the edge (defense in depth above `KNEO_SERV_MAX_BODY_BYTES`)
- IP-based rate limiting if you need it (the service has no built-in per-IP limiter)
- Forwarding the client IP for the service's structured logs

Run the proxy and `kneo-serv` on the same host (or in the same private network) so the unencrypted hop is not exposed.

Bind address

Topology	`--host` value	Rationale
Proxy + service on the same host	`127.0.0.1`	Service is unreachable except through the proxy.
Proxy + service in a shared private network	`0.0.0.0`	The network boundary is the proxy; firewall the service port.
Compose (`compose.yaml`)	`0.0.0.0` inside the container; only the proxy's port is published on the host.	The Compose stack's internal network already isolates the API service.

The Dockerfile defaults to `--host 0.0.0.0 --port 8000`. Override with `KNEO_SERV_PORT` for the published host-side port; the container port is fixed at `8000`.

Trusted-proxy headers

The service logs the immediate TCP peer as `client_ip` in its structured request logs (`observability.md`). When a proxy fronts the service, the immediate peer is the proxy, not the original client. To capture the real client IP in logs and traces, configure the proxy to write `X-Forwarded-For` upstream and ingest it at your log aggregator — the service itself does not rewrite `client_ip` from `X-Forwarded-For` (no implicit trust).

The service does honor `X-Request-ID` and echoes it back on the response. Proxies that already inject a request ID should pass it through; the service generates a UUID per request otherwise.

Reverse-proxy snippets

These are minimal examples. Production configurations should add timeouts, buffer sizing, and rate-limit zones; consult your proxy's docs.

nginx

```
server {
  listen 443 ssl http2;
  server_name kneo.example.com;

  ssl_certificate     /etc/letsencrypt/live/kneo.example.com/fullchain.pem;
  ssl_certificate_key /etc/letsencrypt/live/kneo.example.com/privkey.pem;

  client_max_body_size 2m;   # match or exceed KNEO_SERV_MAX_BODY_BYTES

  location / {
    proxy_pass         http://127.0.0.1:8000;
    proxy_http_version 1.1;
    proxy_set_header   Host              $host;
    proxy_set_header   X-Forwarded-For   $proxy_add_x_forwarded_for;
    proxy_set_header   X-Forwarded-Proto $scheme;
    proxy_set_header   X-Request-ID      $request_id;
    proxy_read_timeout 130s;             # exceed KNEO_SERV_CLIENT_TIMEOUT
  }
}
```

Caddy

```
kneo.example.com {
  reverse_proxy 127.0.0.1:8000 {
    header_up X-Forwarded-For {remote_host}
    header_up X-Request-ID    {http.request.uuid}
  }
  request_body {
    max_size 2MB
  }
}
```

AWS ALB / generic L7 load balancer

- Listener: HTTPS 443 with an ACM certificate; redirect 80 → 443.
- Target group: HTTP, port 8000, healthcheck `GET /readyz` (interval 30s, unhealthy threshold 3, success codes `200`). `/readyz` is unauthenticated by design.
- Idle timeout: ≥ `KNEO_SERV_CLIENT_TIMEOUT` (default 120s); set 130s for a safety margin.
- Body size: ALBs cap at 1 MiB by default — if you accept larger inline specs (`KNEO_SERV_MAX_INLINE_SPEC_BYTES` is 256 KiB by default), use a CloudFront or nginx tier in front and bypass the ALB cap accordingly.

Health-check endpoints behind the proxy

Expose `/livez` and `/readyz` directly to the proxy or load balancer. Both are unauthenticated to keep probe integration simple. Do **not** expose `/readyz` to the public internet — its `503 not_ready` payload includes internal check names and registry contents that should stay inside the operational perimeter.

For most setups: bind the proxy's probe routes to internal listeners only, or restrict the source IP range to your load-balancer subnet.

Verifying TLS is actually in front

```
# TLS terminates at the proxy, service is unreachable directly.
curl -sf https://kneo.example.com/readyz | jq '.metadata.ready'   # → true
curl -sf http://kneo.example.com/readyz                            # → connection refused / 301
curl -sf http://127.0.0.1:8000/readyz                              # → only succeeds from the
proxy host
```

If the third command succeeds from outside the proxy host, the service port is reachable from the public network and the bind address or firewall is misconfigured.

What `kneo-serv` does not provide

- **No built-in TLS.** Terminate at the proxy.
- **No `X-Forwarded-For` rewriting.** Capture client IPs at the proxy or in your log aggregator.
- **No per-IP rate limiting.** Use the proxy's rate-limit zone.
- **No mTLS to upstream providers.** Provider connections go out from the service host; lock down egress at the network layer.

See `security_hardening.md` for the full pre-launch checklist.

Security hardening

Source: *docs/user/security_hardening.md*

Pre-launch checklist for taking a `kneo-serv` deployment to production. Each item references the authoritative configuration doc; this page is the single sheet to walk before going live.

For the auth model itself (roles, scopes, route mapping), see `service_api.md § Authentication`. For audit-event details, see `service_api.md § Audit events`.

Pre-launch checklist

1. Enable authentication

- [ ] `KNEO_SERV_AUTH_ENABLED=true` is set (or `KNEO_SERV_API_KEYS` / `KNEO_SERV_ADMIN_API_KEY` are set, which enables auth implicitly).
- [ ] No API keys are committed to the repo, the Compose `.env` file, or example configs.
- [ ] Each consumer has its own key, named so audit events identify the caller (`KNEO_SERV_API_KEYS='ci:…:service;analyst:…:viewer'`).
- [ ] The admin key (`KNEO_SERV_ADMIN_API_KEY`) is issued separately and used only for break-glass operations.

2. Assign the narrowest role

Pick the narrowest built-in role that covers each caller's needs. The canonical role-to-scope mapping lives in `service_api.md § Authentication`; below is the operational guidance for choosing between them.

Role	Use for
`admin`	Break-glass operator key only
`operator`	Day-to-day operator console / CI deploy
`service`	Server-to-server callers that drive runs
`reviewer`	Human-in-the-loop approvers
`viewer`	Dashboards, read-only analytics

Custom scopes are allowed in the third field of `KNEO_SERV_API_KEYS` when no built-in role fits.

- [ ] No consumer is using `admin` for routine traffic.

- [ ] Read-only consumers are on `viewer`, not `operator`.

3. Rotate keys without downtime

`kneo-serv` has no in-place key rotation API (a secret-manager / rotation surface is deferred to a later major). Rotation is a config swap:

1. Add the new key to `KNEO_SERV_API_KEYS` alongside the old key (semicolon-separated entries; same `name:` is fine).
2. Restart the service. Both keys are now valid.
3. Roll callers over to the new key.
4. Remove the old entry from `KNEO_SERV_API_KEYS`.
5. Restart again.

- [ ] Rotation procedure is rehearsed in staging before production keys are issued.
- [ ] Old keys are revoked, not left "in case."

4. Sign spec bundles for production

For environments that block ad-hoc spec edits:

- [ ] `KNEO_SERV_SPEC_SIGNING_KEY` is set in CI and on the service hosts (different value than any API key).
- [ ] Production deploys use signed bundles only: `kneo spec bundle sign … --approved-by <name> --env prod` and `kneo spec bundle verify <bundle>` in the deploy pipeline.
- [ ] Project config declares `environments.prod.policy_enforcement` so CLI and API spec flows enforce policy after overlays (`project_config.md`).

`spec_path`, `overlays`, and `skills[].source` are filesystem-trusted inputs — they are confined to `KNEO_SERV_SPEC_ROOT`. A request that supplies any of these makes the service read those files from its own filesystem. Two layers guard it:

- The API boundary always rejects `..` parent-traversal and `~` home-expansion in `spec_path`, `overlays`, **and** `skills[].source`.
- Caller-supplied spec / overlay / skill reads — across run, resume, and the `/v1/specs/*` surfaces — are confined to the **spec root**: anything resolving outside it (an absolute path, a traversal, or a symlink escape) is rejected `422 spec_path_confined`.

The spec root is **`KNEO_SERV_SPEC_ROOT`** when set, otherwise the **process working directory**. Confinement is **default-on as of `1.0.0`** (it was opt-in through `0.12.x`, where an absolute path outside the root only logged a `DeprecationWarning`). **Set `KNEO_SERV_SPEC_ROOT` to an explicit allow-listed directory** in any deployment whose specs/skills live outside the working directory, or whose callers are less than fully trusted; otherwise spec reads are confined to the working directory by

default. Prefer inline `spec` (with `overrides`) for less-trusted callers. Spec *content* validation (`/v1/specs/validate`) stays pure (no filesystem I/O beyond the confined spec/overlay/skill read).

Confinement applies to the **service's** reads of caller-supplied paths (the `/v1` surface — the remote-caller threat model). The local `kneo` CLI reads the operator's own filesystem directly and is intentionally **not** confined (operator-trust); when the CLI targets a remote service it sends the resolved spec inline, and the service applies its own confinement.

5. Terminate TLS upstream

- [ ] A reverse proxy in front of the service terminates TLS; the service bind address is `127.0.0.1` or restricted to a private network. See `tls_and_proxy.md`.
- [ ] `/readyz` is exposed only to the load balancer or probe subnet (see `tls_and_proxy.md § Health-check endpoints behind the proxy`).
- [ ] The proxy enforces a request body size limit ≥ `KNEO_SERV_MAX_BODY_BYTES`.

6. Lock down container and host

The bundled Dockerfile already enforces a non-root user (`kneo`); you do not need to override it. (`Dockerfile`)

- [ ] Container runs as the non-root `kneo` user (default).
- [ ] Image is pulled from a trusted registry; tags are pinned by digest in production manifests.
- [ ] CI scans the image for known CVEs; releases blocked on HIGH/CRITICAL findings (§ Image vulnerability scanning).
- [ ] Host or Kubernetes drops Linux capabilities the service doesn't need (no `NET_ADMIN`, no `SYS_ADMIN`).
- [ ] Egress is restricted at the network layer to the provider, MCP, and observability endpoints the deployment actually uses.

The container's filesystem is **not** read-only — the service writes checkpoints, queue state, and optionally SQLite files. If you need read-only-root, mount writable volumes for `.kneo/` (SQLite + continuations) and the artifact paths declared in your spec.

7. Protect the audit trail

- [ ] Audit events are persisted in the same backend as run state (PostgreSQL in production); the DB is backed up per `backup_and_recovery.md`.
- [ ] `audit:read` is scoped to a small set of principals (compliance, on-call, incident-response).
- [ ] Audit-event retention is set deliberately. If `KNEO_SERV_RETENTION_RUNS_DAYS` is set, runs and their audit events age out together — confirm that aligns with your compliance window before enabling it (`environment.md § Retention`).

8. Keep redaction in place

`kneo-serv` redacts secrets, tokens, authorization headers, emails, and SSNs from responses, traces, checkpoints, and CLI JSON output by default (`service_api.md § Redaction`). The two escape hatches both default to off:

- [ ] `KNEO_SERV_OTEL_RECORD_ARGUMENTS=false` (unless tool arguments are classified safe to emit to your trace backend).
- [ ] `KNEO_SERV_OTEL_RECORD_RESULTS=false` (same rationale).
- [ ] Custom tools and middleware do not log user inputs or provider responses without redaction.

Image vulnerability scanning

The release pipeline scans every published GHCR image for known CVEs using Trivy. The scan runs against the **pushed image digest** (the same bytes cosign signed and SBOM attestation describes), so the four supply-chain artifacts — image, cosign signature, SBOM attestation, and scan report — all agree on what they describe.

Locked policy (0.4.0; recorded in archive_TODO-0.4.0.md):

- **Severity threshold:** CVSS≥7 (HIGH and CRITICAL findings).
- **Release-tag scans (`v<version>` / `v<version>rcN`):** blocking. The Trivy step in `release.yml` runs with `--exit-code 1`; HIGH/CRITICAL findings fail the step, preventing the `Publish build artifact` and `Publish GitHub release` steps from running. The publish is the irreversible step, so the gate fires there.
- **PR-time scans:** report-only via `.github/workflows/image-scan.yml`. The same scanner version + severity threshold runs against a locally-built image but with `--exit-code 0`, so findings surface in the PR's check summary without blocking merges. Dev velocity isn't gated on un-fixable transient base-image CVEs; the release gate catches anything that matters before publish.
- **Scan report retention:** the JSON report is attached as a GitHub Actions artifact (`trivy-report-<version>`) on every release-tag build, retained for 90 days. The deployer can download it for audit.

Operator-side verification

Re-run the scan locally against any published tag:

```
trivy image \
  --severity HIGH,CRITICAL \
  --ignore-unfixed=false \
  ghcr.io/kneo-agent/kneo-serv:<tag>
```

Cross-check against the release-time scan output by downloading the `trivy-report-<version>` artifact from the GitHub Release.

Accepted findings

If an upstream CVE has no fix available, or the deployer's risk tolerance accepts a specific finding (e.g. low exploitability in your network posture), record the acceptance in `supply_chain_review.md § Current workspace result` using the same shape as the existing `pip-audit` remediation blocks. For *dependency* (`pip-audit`) findings the release pipeline has no inline-ignore mechanism — those acceptances are deployer policy. For *image* CVEs, the blocking Trivy gate (since 0.4.0) reads acceptances from `.trivyignore` in the repo root, the active image-CVE-acceptance policy.

What `kneo-serv` deliberately does not provide

Operators sometimes go looking for these; document the gap rather than inventing it:

- **No built-in TLS.** Terminate at a reverse proxy (`tls_and_proxy.md`).
- **No per-IP rate limiting.** Use the reverse proxy's rate-limit zone.
- **No mTLS to upstream providers.** Provider HTTPS calls leave the service host; control with egress firewall rules.
- **No live key rotation API.** Keys are configured via env vars; rotate with a config swap and restart (§ 3).
- **No external secret-manager integration.** Secrets are injected via environment variables. Use your platform's secret store (Kubernetes Secrets, AWS Secrets Manager, Vault) to populate the env at startup.
- **No SCIM or directory integration.** API keys are flat-file in `KNEO_SERV_API_KEYS`; map them to identities in your audit log aggregator.

These are tracked in the roadmap, not bugs. See `docs/plan/roadmap.md`.

Observability

Source: `docs/user/observability.md`

Operator guide for wiring `kneo-serv`'s structured logs, request tracing, and OpenTelemetry exports into a production observability stack.

This page is the setup view. For symptoms and recovery when observability itself misbehaves, see `troubleshooting.md § 7`. For the full env-var list, see `environment.md § Observability`.

Three signals, three surfaces

Signal	What it is	Where it comes from
Structured request logs	One JSON record per HTTP request, redacted	`RequestLoggingMiddleware` (always on by default)
Service-side trace events	Per-run trace and checkpoint records, queryable via the API	`TracingMiddleware`, exposed at `/v1/runs/{run_id}/trace`
OpenTelemetry spans	Distributed-tracing spans across SDK-driven agent / tool calls **and** platform-side operations (queue dispatch, worker lease, continuation lock)	`kneo_agent.observability.OpenTelemetryMiddleware` + `kneo_serv.observability.platform_tracer`, opt-in via `KNEO_SERV_OTEL_ENABLED`
Prometheus metrics	Run-queue gauges + per-process run & token counters	`GET /metrics` (since 0.5.0), opt-out via `KNEO_SERV_METRICS_ENABLED`

Prometheus `/metrics`

Since 0.5.0 the service exposes a Prometheus scrape endpoint at `GET /metrics` (root path only — not under `/v1`). It is **unauthenticated**, like `/healthz`: it carries operational counts, not run content or secrets. Mount it only on a network your monitoring stack can reach (bind the service behind a reverse proxy that does not expose `/metrics` publicly), or disable it with `KNEO_SERV_METRICS_ENABLED=false`.

Metric	Type	Meaning
`kneo_runs_started_total`	counter	Runs that began execution.

Metric	Type	Meaning
`kneo_runs_completed_total`	counter	Runs that completed successfully.
`kneo_runs_failed_total`	counter	Runs that failed / timed out / hit max-iterations.
`kneo_runs_dead_lettered_total`	counter	Runs dead-lettered after exceeding `KNEO_SERV_QUEUE_MAX_ATTEMPTS`.
`kneo_tokens_input_total`	counter	Input/prompt tokens consumed across runs (from each run's `metadata["usage"]`, when the runtime reports it).
`kneo_tokens_output_total`	counter	Output/completion tokens produced across runs.
`kneo_tokens_total`	counter	Total tokens (input + output) across runs.
`kneo_runs_queued`	gauge	Runs queued and awaiting a worker (your backlog / backpressure signal).
`kneo_runs_running`	gauge	Runs currently leased by a worker.
`kneo_worker_count`	gauge	Live worker threads in this process.

Counters are **per-process** and reset on restart — use `rate()` in Prometheus. In a multi-process deployment each instance exposes its own counters; aggregate across instances with `sum()`. Latency percentiles are not exported here; read them from the OTel spans or your reverse-proxy access logs.

Structured request logs

Shape

Each request emits a single JSON record on the `kneo_serv.service` logger:

```
{
  "client_ip": "10.0.0.7",
  "duration_ms": 18.214,
  "event": "http_request",
  "method": "POST",
  "path": "/v1/runs",
  "request_id": "f3b3…",
  "run_id": "run_…",
  "status_code": 200,
  "user_agent": "kneo-serv-client/0.2.2"
}
```

Fields the middleware always emits: `event`, `request_id`, `method`, `path`, `status_code`, `duration_ms`. Optional fields when available: `client_ip`, `user_agent`. Route-derived fields when the path includes them: `run_id`, `continuation_id`. When the request raises: `error` (exception class name) and `message` (exception message). Redaction is applied to every payload before it reaches the log line. (`kneo_serv/observability/structured_logging.py`)

Configuration

Variable	Default	Purpose
`KNEO_SERV_REQUEST_LOGS`	`true`	Enable the JSON request log middleware.
`KNEO_SERV_LOG_LEVEL`	`INFO`	Stack-wide level: `kneo_serv.service` + `kneo_serv.platform` + `kneo_agent` SDK.

`request_id` is generated server-side as a UUID unless the client sends `X-Request-ID`; either way the service echoes it back on the response header.

Production tuning

- Keep `KNEO_SERV_LOG_LEVEL=INFO` in production. `DEBUG` doubles log volume and can leak diagnostic payloads from middleware that wraps the request logger. Note it is **stack-wide**: it sets the request logger, the platform worker / lease / drain logger (`kneo_serv.platform`), and the `kneo_agent` SDK logger together, so `DEBUG` turns all three up at once — useful for diagnosing a stuck run end-to-end, noisy as a default.
- Configure your container runtime's log driver (Docker `json-file` with rotation, Kubernetes `kubectl logs` rotation, journald) — the service writes to stdout and relies on the runtime to rotate.

Log aggregation wiring

- **ELK / OpenSearch.** Ship stdout via Filebeat or Vector. The records are already JSON; map `request_id` and `run_id` as indexed fields. Pin `service.name=kneo-serv` from the shipper for cross-deployment search.
- **Loki.** A Promtail pipeline with a `json` stage will lift `request_id`, `run_id`, `status_code`, and `duration_ms` to labels. Keep label cardinality bounded — don't promote `request_id` to a Loki label, query it as content.
- **Cloud-managed (CloudWatch Logs, GCP Logging).** Forward stdout; the managed pipeline parses JSON automatically.

The reverse proxy in front of `kneo-serv` (`tls_and_proxy.md`) has the true client IP. The service logs the immediate TCP peer; correlate to the proxy's access logs by `request_id` (forward `X-Request-ID` upstream).

Service-side trace events

Service-side trace events are persisted as part of run state and returned at `GET /v1/runs/{run_id}/trace`. They cover workflow step transitions, tool calls, checkpoints, and audit boundaries. These events are emitted by `TracingMiddleware` independent of any OTel exporter, so they are always available even without OpenTelemetry.

See `service_api.md § Audit events` and `service_api.md § Replay and checkpoint diff` for the contract.

Audit-event export

Audit events are always queryable via `GET /v1/audit-events` and stored in the run-state store. For compliance retention you can additionally **stream them out**: set `KNEO_SERV_AUDIT_EXPORT_ENABLED=true` and each persisted event is emitted as a JSON line on the dedicated `kneo_serv.audit` logger, from the same `record_audit_event` chokepoint (so the payload is already redacted).

It is **off by default** — enabling it is a zero-behavior-change opt-in. The transport is plain stdlib logging, so wire it to your sink the usual way:

```
 import logging
from logging.handlers import SysLogHandler

audit = logging.getLogger("kneo_serv.audit")
audit.addHandler(SysLogHandler(address=("siem.internal", 514)))
audit.setLevel(logging.INFO)
```

With no dedicated handler the events propagate to the root logger and land in your normal JSON logs alongside the request logs. This is the v1 sink; a direct SIEM/OTLP-logs exporter may follow.

OpenTelemetry spans

When the deployment includes the SDK telemetry support (the `[telemetry]` or `[deploy]` extras), set `KNEO_SERV_OTEL_ENABLED=true` to attach `kneo_agent.observability.OpenTelemetryMiddleware`. Argument and result capture (`KNEO_SERV_OTEL_RECORD_ARGUMENTS`, `KNEO_SERV_OTEL_RECORD_RESULTS`) are off by default because tool inputs and outputs frequently contain user payloads — enable them only after the deployment's data classification has approved payload capture.

See `environment.md § Observability` for the full env-var reference.

Platform-side spans

The SDK's `OpenTelemetryMiddleware` covers the *agent boundary* — runs, tool calls, model calls. The platform also instruments operations that happen *outside* the agent's execution:

Span name	Where	Attributes
`kneo.queue.dispatch`	`PlatformManager.dispatch_run` — when a run is enqueued for an async worker	`kneo.run.id`
`kneo.worker.lease`	Async worker loop — one span per lease attempt against the queue	`kneo.worker.id`, `kneo.worker.lease_seconds`, `kneo.worker.claimed` (bool), `kneo.run.id` (if claimed)
`kneo.continuation.lock`	`PlatformManager.resume_human_task` — when the per-continuation lock is acquired before resume	`kneo.continuation.id`, `kneo.lock.name`, `kneo.lock.ttl_seconds`, `kneo.lock.acquired` (bool)

These spans share the same `KNEO_SERV_OTEL_ENABLED` flag — they're a clean no-op when telemetry is off (no overhead beyond a single env-var check). Span names use the `kneo.<area>.<operation>` convention so they sort cleanly alongside SDK-owned spans in tracing UIs.

GenAI semantic conventions (`gen_ai.*`). The OpenTelemetry GenAI attributes (`gen_ai.system`, `gen_ai.operation.name`, token counts, …) are emitted by the SDK's `OpenTelemetryMiddleware` on the **agent / model-call** spans — that is where the model provider is known. The platform-side spans

above are provider-agnostic infrastructure operations (queue / lease / lock) and deliberately do **not** carry `gen_ai.*`; querying GenAI telemetry uses the SDK-owned model spans.

Lease spans with `kneo.worker.claimed=false` indicate an empty queue — useful for measuring how often workers idle. Continuation lock spans with `kneo.lock.acquired=false` correlate with the `LockAcquisitionError` shown in troubleshooting.md § 8.1.

Exporter configuration

The service uses the OpenTelemetry global tracer provider; exporters are configured with standard `OTEL_*` environment variables that the OTel SDK reads. Example for OTLP/HTTP to any compatible backend (Honeycomb, Grafana Tempo, Tempo Cloud, Datadog, an OTel Collector):

```
export KNEO_SERV_OTEL_ENABLED=true

export OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf
export OTEL_EXPORTER_OTLP_ENDPOINT=https://api.honeycomb.io
export OTEL_EXPORTER_OTLP_HEADERS="x-honeycomb-team=$HONEYCOMB_API_KEY"
export OTEL_SERVICE_NAME=kneo-serv
export OTEL_RESOURCE_ATTRIBUTES="deployment.environment=prod"
```

For a self-hosted OTel Collector running as a sidecar:

```
export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317
export OTEL_EXPORTER_OTLP_PROTOCOL=grpc
```

If OTel does not appear to be exporting, see `troubleshooting.md § 7.2`.

What to watch in production

A minimal alerting baseline covers the failure modes that page on-call:

Signal	What it means	Where to read it
`/readyz` returns `503` for more than 1 probe interval	A dependency check is failing	Reverse proxy / load balancer health checks
Sustained `5xx` rate above baseline	Service-side errors	Proxy access logs; `status_code` from JSON logs
`duration_ms` p95 climbing over baseline	Latency regression — provider, queue, or DB pressure	JSON log records

Signal	What it means	Where to read it
Queue depth (`status=queued`) growing unbounded	Workers stuck or backpressured	`/readyz` `queue` check; `curl "$BASE/v1/runs?status=queued&limit=20"`
Spike in `event=http_request` records with `error`	Application-level exceptions	JSON log records

Wire your alerting against these signals from the proxy and the aggregated logs; the service does not push its own alerts.

What this page does not cover

- **Per-IP rate limiting and traffic shaping.** The reverse proxy's job (`tls_and_proxy.md`).
- **Request-level latency histograms on `/metrics`.** The `/metrics` endpoint (since 0.5.0) exports run-queue gauges and run counters, not per-request latency percentiles. Derive request latency from OTel spans or the reverse-proxy access logs.
- **Tracing internals.** For the design of the in-process tracer and checkpoint events, see `docs/dev/design.md` and `docs/dev/implementation_map.md` .

Monitoring & alerting

Source: `docs/user/monitoring_and_alerting.md`

observability.md covers the *setup* — the three signals (Prometheus `/metrics`, structured logs, OTel spans) and how to wire them. This page is the *interpretation*: what to alert on, the expression to alert with, what a firing alert means, and what to do about it. It assumes the `/metrics` endpoint is scraped (since 0.5.0) and the JSON request logs are aggregated.

> Counters on `/metrics` are **per-process** and reset on restart — always wrap them in `rate()`, and `sum()` across instances in a multi-process deployment. The metric surface is the table in observability.md § Prometheus `/metrics`.

Alert catalogue

Each row is a *page-worthy* signal. PromQL is illustrative — tune the thresholds and windows to your traffic and SLOs.

Queue backlog / backpressure

```
# Backlog growing and not draining (tune 50 / 10m to your throughput)
kneo_runs_queued > 50 and deriv(kneo_runs_queued[10m]) > 0
```

Means: runs are arriving faster than the worker pool drains them, or workers are wedged. **Do:** check `kneo_worker_count` is non-zero and `kneo_runs_running` is moving; scale `KNEO_SERV_WORKER_CONCURRENCY` or add instances; if a deployment runs with `KNEO_SERV_MAX_QUEUE_DEPTH` set, sustained backlog at the cap means clients are getting `503` load-shed (those runs are terminalized `failed{queue_full}`, not silently dropped).

Worker starvation

```
kneo_worker_count == 0 and kneo_runs_queued > 0
```

Means: queued work with no live worker to claim it. Workers poll persistently until shutdown (0.10.0), so a zero count with a backlog points at a crashed pool or a process that never called `start_worker`. **Do:** check the process is up and `/readyz` is green; restart drains the queue (leases are reclaimed).

Failure & dead-letter rate

```
# Failures as a fraction of completions over 5m
rate(kneo_runs_failed_total[5m])
```

```
  / clamp_min(rate(kneo_runs_completed_total[5m]), 1) > 0.1

# Any dead-lettering is worth attention
rate(kneo_runs_dead_lettered_total[5m]) > 0
```

Means: a rising failed-ratio is a provider outage, a bad spec, or store pressure; **dead-lettering** means a run exceeded `KNEO_SERV_QUEUE_MAX_ATTEMPTS` re-claims (it repeatedly crashed its worker). **Do:** read the failing runs' traces (`GET /v1/runs/{id}/trace`) and the JSON logs for the exception; a dead-letter usually means a poison run — fix the spec/tool, don't just raise the cap.

Latency regression

```
histogram_quantile(0.95, sum by (le) (rate(<proxy_request_duration_bucket>[5m]))) > <slo>
```

`/metrics` exports run counters and queue gauges, **not** request-latency histograms — read p95 from the reverse-proxy access logs or the OTel spans, or from the `duration_ms` field on the JSON request logs. **Means:** provider slowness, queue wait, or DB pressure. **Do:** correlate with `kneo_runs_queued` (queue wait) and provider latency in the spans.

Token spend

```
rate(kneo_tokens_total[1h]) > <budget_per_hour>
```

Means: consumption above your cost envelope. **Do:** the per-run hard ceiling is `TokenBudgetMiddleware` (a run that hits it ends `failed{token_budget_exceeded}`); this alert catches *aggregate* spend trending over budget across runs.

Readiness / dependency health

```
# From the proxy/LB health checks, not /metrics:
/readyz returns 503 for more than one probe interval
```

Means: a dependency probe (store, queue, configured provider secret) is failing — `/readyz` reports per-check detail. **Do:** read the `checks` block in the `/readyz` body; see incident_response.md.

Signals that aren't on `/metrics`

Two state-growth signals matter operationally but are read from the API / store, not the scrape endpoint (see checkpoint_and_state_lifecycle.md):

- **Continuation backlog** — `GET /v1/human-tasks` (or a `COUNT` on the continuation store). A count that only grows means human tasks are neither answered nor expired. Give human tasks an `on_timeout` and run `prune_expired_human_tasks`.
- **Checkpoint / store growth** — track store size or a checkpoint `COUNT`. Rising size with a flat run rate means the retention sweep (`prune_retention`) isn't running. Note the 0.10.0 liveness guard *intentionally* retains live (blocked/running) runs' checkpoints regardless of age, so a large blocked-run population is expected store.

Wiring

The service does not push its own alerts — it exposes signals. Point your Prometheus/Alertmanager at `/metrics`, your log pipeline at the JSON records, and your proxy's health checks at `/readyz`. Keep `/metrics` unauthenticated but **not publicly reachable** (it carries operational counts, no run content); bind it behind the reverse proxy per tls_and_proxy.md.

Checkpoint & state lifecycle

Source: `docs/user/checkpoint_and_state_lifecycle.md`

An operator's guide to the durable state a run accumulates — checkpoints, trace events, and human-task continuations — what writes it, how it grows, when retention prunes it, and how to read it. For *what each run status means* see run_lifecycle.md; for *backup/restore* of the whole store see backup_and_recovery.md.

What a run persists

Every run writes three kinds of durable state to the configured store (SQLite or PostgreSQL — see deployment.md):

Record	What it is	Why it exists
Run row	The `RunState`: status, output, error, deadline, the redacted final trace.	The authoritative run record `GET /v1/runs/{id}` returns.
Checkpoints	Per-step snapshots written as the run progresses, each carrying the **trace delta** (events since the previous checkpoint) plus a redacted state snapshot.	Resume/replay: a paused or interrupted run is rebuilt from its checkpoints; `GET /v1/runs/{id}/trace` reassembles the timeline from them.
Continuations	The paused-run record for a human task (the pending request + the message thread).	A `blocked` run resumes from its continuation when the human responds.

Checkpoints and trace events are **redacted at write time** — secrets and PII never land in the persisted snapshot (see observability.md and security_hardening.md).

How it accumulates

- **Checkpoints** are appended per workflow step / agent iteration. A long sequential workflow or a high-iteration agent writes one checkpoint per step, so checkpoint volume scales with *steps* × *runs*, not wall-clock.
- **Trace events** within a single run's live buffer are bounded by `KNEO_SERV_TRACE_MAX_EVENTS` (default **10 000**; `0` disables the cap). Past the cap, further events are *counted but not buffered* — the full timeline still reassembles from the checkpoints

via `GET /v1/runs/{id}/trace`, which merges checkpoint deltas with the run's events. A resumed run trims the *oldest* seed events from its live buffer to reserve headroom for the new leg; those trimmed events are recovered from the prior-leg checkpoints on read.
- **Continuations** accumulate one per `blocked` run (a run waiting on a human task). They persist until the run resumes, is cancelled, times out, or the human task expires.

Reading it

Endpoint	Returns
`GET /v1/runs/{id}/trace`	The full OTel-style trace, merged from checkpoints + the run's events (deduped by `event_id`).
`GET /v1/runs/{id}/checkpoints`	The raw checkpoint list for the run.
`GET /v1/runs/{id}/checkpoints/diff`	A diff view between checkpoints (replay/debug).
`GET /v1/human-tasks`	Continuations currently waiting on a human task.

Retention — what prunes, and what is protected

State does **not** grow without bound *if* you run the retention sweep. Retention is operator-driven (`prune_retention` / the maintenance pass — no built-in scheduler); each category has its own window on the `RetentionPolicy`:

Knob	Prunes
`runs_days`	Terminal run rows older than the window.
`checkpoints_days`	Checkpoints older than the window **whose run has reached a terminal status** — see the liveness guard below.
`continuations_days`	Continuation records older than the window.
`queue_days`	Completed/failed queue records.
`audit_days`	Audit events.

Checkpoint liveness guard (0.10.0). `checkpoints_days` prunes by age **only for runs that are already terminal** (completed / failed / cancelled / timed_out / expired) or whose run row no longer exists. The checkpoints of a *live* run — `running`, `blocked`, `created`, `paused` — are **retained regardless of age**, so a run that has been blocked on a human task for longer than

`checkpoints_days` keeps the checkpoints it needs to resume. (Before 0.10.0, checkpoints were pruned purely by age, which could delete the resume state out from under a long-paused run.)

Expired human tasks. `prune_expired_human_tasks` transitions a human-blocked run to `expired` once its task deadline passes (or applies its `on_timeout` policy), and clears the stale continuation. Run it on a cadence so abandoned approvals don't pin continuations and their checkpoints forever. A run driven to a terminal status this way then becomes eligible for the age-based checkpoint prune above.

Operating guidance

- **Set a retention policy.** Without one, terminal runs' checkpoints and continuations persist for the life of the store. A typical on-prem policy keeps a few weeks of history; size it against your run volume × steps-per-run and your disk.
- **Run the sweeps on a schedule.** Wire `prune_retention` and `prune_expired_human_tasks` into cron / a sidecar (the service ships no scheduler). Pair them: expiring stale human tasks first makes their runs terminal, which then lets the checkpoint prune reclaim their state.
- **Watch continuation backlog and checkpoint growth** — see monitoring_and_alerting.md. A continuation count that only grows means human tasks are never being answered *or* expired; rising store size with a flat run rate means the retention sweep isn't running.
- **Long-lived human-in-the-loop workflows** keep their continuation and checkpoints for as long as they stay `blocked` — that is by design (the liveness guard), but it means an unbounded population of never-answered approvals is unbounded state. Give human tasks a timeout (`on_timeout`) so they can't pile up indefinitely.

Performance and capacity

Source: `docs/user/performance.md`

This page is about **platform overhead and capacity planning** — how much throughput a `kneo-serv` deployment sustains, where the latency goes, and which knobs move the numbers. It is *not* about model quality or provider latency: those dominate real run wall-time and are outside the service's control.

The guidance here is anchored to a repeatable bench harness (`scripts/bench`) so you can reproduce the numbers on your own hardware rather than trusting a table you can't audit. **Do that before you size a production deployment** — the illustrative numbers below come from a single modest machine and are meant to show *shape and ratios*, not to be copied into a capacity plan.

What determines run throughput

A run's wall-time is the sum of:

1. **Provider/model latency** — the LLM call(s). Usually 100 ms–10 s+ per step and the dominant term for real workloads. The service does not control this; tune it with `KNEO_SERV_PROVIDER_TIMEOUT_SECONDS` / `KNEO_SERV_PROVIDER_RETRIES` (see environment.md).
2. **Platform overhead** — compile-from-spec, queue dispatch, worker lease, state save, checkpoint append. This is what the service *can* control and what the bench harness isolates by running an echo agent with no provider I/O.
3. **Persistence latency** — the per-save cost of the state store. SQLite is a single-writer embedded file; PostgreSQL supports concurrent writers. This is the single biggest capacity lever (see below).

When you read "throughput" below it means **runs/second of platform overhead** — the ceiling you hit if the model were instantaneous. Real throughput is `min(platform_ceiling, provider_ceiling)`, and for most deployments the provider is the binding constraint.

The bench harness

`scripts/bench` drives the real `PlatformManager` with a deterministic echo agent and reports throughput, latency percentiles, and peak RSS. Run it as a module:

```
 # 300 runs, 8 concurrent workers, SQLite store, synchronous execute path
python -m scripts.bench --total-runs 300 --concurrency 8 --store sqlite
```

```
# Machine-readable line for sweep aggregation
python -m scripts.bench --total-runs 300 --concurrency 8 --store sqlite --json
```

Key options:

Option	Default	Meaning
`--total-runs`	200	Measured runs (after warmup).
`--concurrency`	8	Worker threads (sync mode) / dispatch fan-out.
`--store`	`sqlite`	`sqlite`, `memory`, or `postgres`.
`--mode`	`sync`	`sync` (thread pool of `execute_run`) or `queue` (dispatch + worker drain).
`--agent-delay`	0.0	Simulated provider latency per run (seconds).
`--postgres-dsn`	—	Required for `--store postgres`.

`sync` mode measures the execution + persistence path with clean per-run latency. `queue` mode measures end-to-end durable-queue throughput, so its per-run latency includes queue wait — useful for understanding worker-drain behaviour, not for comparing per-run cost.

To benchmark PostgreSQL (the production-representative path):

```
python -m scripts.bench --store postgres \
  --postgres-dsn "postgresql://kneo:kneo@localhost:5432/kneo" \
  --total-runs 300 --concurrency 8 --json
```

The harness is also exercised by `pytest -m bench` as a smoke check so it does not bit-rot; that lane asserts the harness runs and returns sane metrics, it is **not** a performance gate.

Reference profile and measured numbers

!!! warning "A reference baseline, not a production sizing table" Measured on a **dedicated bare-metal host** (2026-06-18): AMD Ryzen Threadripper PRO 3975WX (64 logical cores), **126 GiB RAM**, Linux 6.17 (x86_64), CPython 3.12.3; SQLite on local disk and PostgreSQL 16.14; **echo agent, zero provider delay** — so these are the *platform-overhead ceiling* (the rate you'd hit if the model were

instantaneous), not your real throughput. Single run, `--total-runs 500`, 50-run warmup, 256-byte payload. They show *ratios and shape*, not a sizing table — **re-run the harness on your own hardware and store before sizing** (the release procedure that automates this is bench_soak_runbook.md, and the operator how-to is dev/release_soak.md).

Store / mode	Concurrency	Throughput (runs/s)	p50	p95	p99	Peak RSS
sqlite / sync	1	51	10.7 ms	11.6 ms	12.4 ms	56 MiB
sqlite / sync	4	51	54.5 ms	71.7 ms	84.0 ms	56 MiB
sqlite / sync	8	50	105.9 ms	170.4 ms	230.3 ms	57 MiB
sqlite / sync	16	51	197.3 ms	415.9 ms	634.9 ms	58 MiB
postgres / sync	1	79	8.1 ms	8.6 ms	8.9 ms	57 MiB
postgres / sync	8	77	72.3 ms	99.6 ms	139.0 ms	58 MiB
postgres / sync	16	76	149.9 ms	220.5 ms	324.6 ms	58 MiB
postgres / sync	32	75	260.4 ms	626.5 ms	804.5 ms	57 MiB
postgres / queue	8	44	5039 ms	5830 ms	6233 ms	57 MiB

With a simulated 0.5 s provider delay (postgres / sync, 8 concurrent) throughput is 15 runs/s at p50 513 ms — the ~13 ms of platform overhead is dwarfed by model latency, which is the point: real throughput is provider-bound.

What this profile shows — and what should hold directionally on any hardware:

- **SQLite throughput is flat as concurrency rises** (~51 runs/s from 1 → 16 here) while per-run latency scales linearly (p50 10.7 → 197 ms) — SQLite is a single writer, so concurrent runs serialize on the write lock. **More workers do not buy more write throughput on SQLite**; they buy queueing. This is the headline capacity fact. (The *absolute* SQLite ceiling is fsync/disk-bound and varies by hardware — it is lower here than on a laptop with a faster single-thread fsync — but the flat-with-concurrency shape is invariant.)
- **PostgreSQL holds ~75–79 runs/s flat from 1 → 32 concurrent** (sync): per-run latency scales linearly (more in-flight) but *throughput does not collapse*, and on this host PostgreSQL

out-throughputs SQLite even at concurrency 1. This is the measured evidence for the "move to PostgreSQL for write-concurrent deployments" guidance below — the concurrent-writer path scales where the single-writer one does not.
- **Queue mode latency is dominated by queue wait** (p50 ~5 s here): the background pool drains FIFO at the default one worker per process, so the ~44 runs/s is the single-worker *drain rate*, not per-run cost. Add workers / processes to raise it.
- **Peak RSS is flat (~56–59 MiB)** across every shape — memory is not the first constraint at these volumes; the write path is.

Sustained-load soak (resource stability)

A **production-class, bare-metal sustained run** on the host above (AMD Threadripper PRO 3975WX, 64 cores, 126 GiB, PostgreSQL 16.14) — **1 hour, 16 workers, multi-worker queue drain** — held steady end to end (2026-06-18):

- **158,745 runs dispatched, 0 errors**; backlog bounded (peak queued 59) and drained to zero on stop.
- **RSS 61.3 → 61.8 MiB (+0.8%)** over the full hour (peak 61.8 MiB), inside a tightened 10% tolerance — no leak in the worker pool, the cancellation-Event map, the MCP session host, or event loops under sustained load.
- **Thread count bounded at 32** — the persistent idle-poll worker (0.10.0) neither spawns nor leaks threads.

Run in the same pass against the same PostgreSQL: **queue-depth backpressure** (load-sheds `QueueFullError`, terminalizes the rejected run `failed{queue_full}` with no phantom row, drops its cancellation Event) and the reliability test lanes — `postgres_integration` (CAS terminal writes, cross-process double-claim / idempotency, prune liveness), operability/durable-queue, and MCP transports — all passed. The procedure is in bench_soak_runbook.md; run it with dev/release_soak.md.

Minimum sizing (a starting point)

The numbers above are reference baselines, not a guarantee for your workload — but they do bound the **service process's own footprint**, which is the part you can size confidently:

- **Memory** — peak RSS is flat at **~57–62 MiB** across every bench shape and held at **61.3 → 61.8 MiB over the 1-hour / 158,745-run soak** (no leak). Memory is not the bottleneck. **Start with 512 MiB** for the service container (≈8× the measured peak) and only revisit if your own profile shows otherwise.
- **CPU** — a run's wall-clock is dominated by the model provider's network latency, not service CPU; platform overhead per run is sub-millisecond-to-low-ms (see the table above). **Start**

with 1 vCPU for a single-team deployment; add cores/**processes** (not just threads) when you need write-concurrent throughput on PostgreSQL.

- **Workers** — one process runs `KNEO_SERV_WORKER_CONCURRENCY` worker threads (the soak ran 16). Raise it for more in-flight runs, bounded by your provider's rate limits; for write-concurrent scale beyond one process, run multiple service processes against shared PostgreSQL (see deployment.md § Workers, scaling).
- **Persistence** — SQLite suits a single node at low write concurrency (throughput is flat ~51 runs/s — single writer); use PostgreSQL for concurrent runs / multiple workers, and size it for your retention window and connection count. **This sizing covers the service process only** — budget your PostgreSQL (and any provider-side resources) separately.

This is a floor to deploy against, then **measure on your own hardware** with the bench harness before committing capacity (see the reference-profile warning above).

Choosing a store for capacity

	SQLite	PostgreSQL
Writers	Single (serialized)	Concurrent
Concurrency scaling	Throughput flat-to-down	Scales with connections/cores
Durability	File `fsync`	WAL + replication (your responsibility)
When	Single-node, low write concurrency, simplest ops	Concurrent runs, multi-worker, production

If your bench shows the SQLite write lock is your ceiling and you need more concurrent run throughput, **move to PostgreSQL** — that is the supported path for write-concurrent deployments. See `tutorial_postgres_deployment.md` for the guided setup. (The PostgreSQL store-contract + multi-connection concurrency suite already runs as a default CI lane on every PR — promoted in 0.6.0; see ga_notes.md.)

PostgreSQL sizing notes

Connection pooling lives at the psycopg layer; size the pool to your worker concurrency plus headroom for the API request path. Start from your bench: run `--store postgres` at the concurrency you intend to deploy, watch p95/p99, and provision database CPU and `max_connections` so the store is not the binding constraint. Replication and cross-region failover are the deployer's responsibility and an explicit non-goal.

Capacity tuning knobs

These environment variables move the platform-overhead and storage-growth terms. Full semantics in environment.md; the deployment-oriented subset and defaults also appear in `tutorial_postgres_deployment.md § 8`.

The **Default** column is the code default (no env set); the **Tune when** column carries a suggested production value where it differs.

Variable	Default	Tune when
`KNEO_SERV_PROVIDER_TIMEOUT_SECONDS`	unset (no timeout)	Provider tail latency needs bounding — `120` is a reasonable start.
`KNEO_SERV_PROVIDER_RETRIES`	`0`	Provider has a documented transient error rate — `2` is a reasonable start.
`KNEO_SERV_MAX_BODY_BYTES`	1 MiB	Larger inline specs or override payloads.
`KNEO_SERV_MAX_INPUT_CHARS`	20000	Run inputs larger than the default.
`KNEO_SERV_RETENTION_RUNS_DAYS`	unset	Cap run-history storage growth.
`KNEO_SERV_RETENTION_CHECKPOINTS_DAYS`	unset	Cap checkpoint-history storage growth.
`KNEO_SERV_RETENTION_QUEUE_DAYS`	unset	Cap queue-record storage growth.
`KNEO_SERV_CHECKPOINT_COMPRESS_BYTES`	64 KiB	Many large checkpoints; lower to compress more aggressively.
`KNEO_SERV_CHECKPOINT_MAX_BYTES`	—	Bound a single checkpoint payload.

Checkpoint payload growth

Branch-level checkpointing (concurrent and group-chat workflows) emits one checkpoint per step occurrence, so checkpoint volume grows with workflow breadth × rounds, not just run count. Each payload over `KNEO_SERV_CHECKPOINT_COMPRESS_BYTES` is compressed;

`KNEO_SERV_CHECKPOINT_MAX_BYTES` bounds a single payload. For long-lived deployments set `KNEO_SERV_RETENTION_CHECKPOINTS_DAYS` so checkpoint history is pruned. The `step_iteration_counts` and `execution_context` metadata the run carries is bounded by step count and does not grow unboundedly within a run.

Worker concurrency and queue-lease tuning

Durable-queue runs are leased FIFO by the worker pool. By default there is **one worker thread per process** (`KNEO_SERV_WORKER_CONCURRENCY=1`), so queue throughput is a single drain rate — the `queue` -mode bench above shows the resulting per-run queue wait. Raise `KNEO_SERV_WORKER_CONCURRENCY` for **provider-bound** workloads: the threads overlap on provider I/O (the LLM call) while the shared store connection is held only briefly, so N workers drain roughly N× faster until the store write path saturates. For **store-bound** workloads on SQLite the single-writer lock caps the gain (see the table above) — move to PostgreSQL, where multiple worker *processes* claim safely via `FOR UPDATE SKIP LOCKED`, for write-concurrent horizontal scale.

`KNEO_SERV_WORKER_LEASE_SECONDS` (default 300) sets how long a claimed run is leased; a worker that dies mid-run has its run re-claimed once the lease expires (bounded by the `KNEO_SERV_QUEUE_MAX_ATTEMPTS` dead-letter cap). Set `KNEO_SERV_MAX_QUEUE_DEPTH` to shed load with `503` once the backlog reaches a ceiling instead of growing the queue unboundedly.

Watch the backlog via the `kneo_runs_queued` / `kneo_runs_running` / `kneo_worker_count` gauges on the Prometheus `/metrics` endpoint, and dispatch/lease latency via the `kneo.queue.dispatch` and `kneo.worker.lease` OpenTelemetry spans. Rising `kneo_runs_queued` with workers busy means the run path — not the API — is your constraint: add worker concurrency or move to PostgreSQL.

A capacity-planning recipe

1. Pick the store you will deploy (PostgreSQL for any concurrent-write load).
2. Run the bench at your intended concurrency with `--agent-delay` set to your provider's typical per-step latency, so the numbers reflect real run shape: `python -m scripts.bench --store postgres --postgres-dsn ... --concurrency 16 --agent-delay 1.0 --json` .
3. Read throughput and p95/p99. If the store is the ceiling, scale database resources or reduce write concurrency; if the provider is the ceiling, the platform has headroom.
4. Record the hardware profile alongside the numbers — a throughput figure without a profile is not reproducible and will mislead the next operator.
5. Set retention env vars so storage growth is bounded for the run/checkpoint volume you measured.

See also

- Deployment — supported deployment shapes.
- Environment variables — full knob reference.
- Observability — the spans to watch for capacity signals.
- PostgreSQL deployment tutorial — guided setup for the write-concurrent path.
- Backup and recovery — durability operations.

Backup and recovery

Source: *docs/user/backup_and_recovery.md*

Production procedure for backing up `kneo-serv` state, verifying restores, and rolling back a deployment. This page consolidates the operator surface; the underlying Python API and SQL commands stay in their respective references.

For the upgrade context that ends in "and keep a backup", see `upgrade.md`. For the Python backup API used by the SQLite maintenance helpers, see `service_api.md § Backup and restore`.

What needs to be preserved

State	Where it lives	Backup mechanism
Run state, queue, checkpoints, audit events, idempotency, locks, policies	PostgreSQL (`KNEO_SERV_DATABASE_URL` set) or SQLite at `.kneo/kneo_runs.sqlite` (default)	`pg_dump` / SQLite online-backup
Workflow continuations	PostgreSQL when set, otherwise files under `.kneo/continuations/`	DB dump or filesystem backup
Spec bundles	Source repo + your CI artifacts (signed bundles)	Repo + artifact store
Artifacts (workflow outputs)	Filesystem paths declared by your specs	Filesystem backup
Logs	stdout via container log driver → log aggregator	Aggregator retention

The DB is the load-bearing piece. Everything else can be reconstructed from the DB and your spec repo, except for filesystem-stored continuations and artifacts when PostgreSQL is not configured.

PostgreSQL — production path

The Compose stack and any production deployment should set `KNEO_SERV_DATABASE_URL`. In that mode all state above (except artifacts) lives in PostgreSQL.

Take a backup

```
docker compose --env-file deploy/production.env exec db \
  pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB" \
  | gzip > "kneo_serv-$(date +%Y%m%d-%H%M).sql.gz"
```

For a host-level Postgres install, run `pg_dump` directly as the `postgres` user; the data shape is the same.

Restore from a backup *(destructive — wipes current state)*

```
gunzip -c kneo_serv-YYYYmmDD-HHMM.sql.gz \
  | docker compose --env-file deploy/production.env exec -T db \
      psql -U "$POSTGRES_USER" "$POSTGRES_DB"
```

Restore replaces every row in the database. Stop the API container first so no in-flight write races the restore:

```
docker compose --env-file deploy/production.env stop api
gunzip -c kneo_serv-YYYYmmDD-HHMM.sql.gz \
  | docker compose --env-file deploy/production.env exec -T db \
      psql -U "$POSTGRES_USER" "$POSTGRES_DB"
docker compose --env-file deploy/production.env start api
```

Off-site rotation

Local backups protect against operator error, not host loss. After each dump, copy the gzip off the host:

- S3, Azure Blob, or GCS bucket with versioning + lifecycle to archive older dumps.
- Encrypt at rest (server-side encryption is sufficient if your control plane is locked down; client-side encryption for stricter regimes).
- Apply a separate IAM identity for upload-only versus read.

Data-only restore into a clean volume

For test-restore drills and disaster recovery into a fresh PostgreSQL volume, the service handles schema migrations on startup. Capture a data-only dump and exclude the `schema_migrations` rows so the new volume's migration state isn't overwritten:

```
docker compose --env-file deploy/production.env exec -T db \
  pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" --data-only --inserts \
  -f /tmp/kneo_serv_data.sql
docker cp <db-container-id>:/tmp/kneo_serv_data.sql /tmp/kneo_serv_data.sql
```

```
grep -v "INSERT INTO public.schema_migrations" /tmp/kneo_serv_data.sql \
  > /tmp/kneo_serv_data_restore.sql
```

Restore into a clean volume after the API has come up at least once (so migrations have run):

```
 docker compose --env-file deploy/production.env down -v
docker compose --env-file deploy/production.env up --build -d
docker cp /tmp/kneo_serv_data_restore.sql \
  <db-container-id>:/tmp/kneo_serv_data_restore.sql
docker compose --env-file deploy/production.env exec -T db \
  psql -v ON_ERROR_STOP=1 -U "$POSTGRES_USER" -d "$POSTGRES_DB" \
  -f /tmp/kneo_serv_data_restore.sql
docker compose --env-file deploy/production.env restart api
```

`ON_ERROR_STOP=1` aborts the restore on the first failing INSERT so you don't end up with partial state.

SQLite — single-host installs

When `KNEO_SERV_DATABASE_URL` is unset, run state lives in `.kneo/kneo_runs.sqlite` and continuations in `.kneo/continuations/`. The service ships an online backup helper:

```
 from kneo_serv.maintenance import backup_sqlite_database, restore_sqlite_database

# Online — safe while the service is running
backup_sqlite_database(
    ".kneo/kneo_runs.sqlite",
    ".kneo/backups/kneo_runs-2026-05-12.sqlite",
)

# Restore into a new location, then swap into place during a window
restore_sqlite_database(
    ".kneo/backups/kneo_runs-2026-05-12.sqlite",
    ".kneo/kneo_runs.restored.sqlite",
)
```

`backup_sqlite_database` uses SQLite's `backup()` API and is safe to run against a live database. `restore_sqlite_database` is a plain file copy — stop the service before swapping the restored file into the live path, or you'll race the writer.

Also back up `.kneo/continuations/` and any artifact paths your specs write to; these are not inside the SQLite file.

Backup frequency

There is no single recommended cadence. Tie it to your retention policy and your tolerance for re-running work:

Workload shape	Cadence
Low run volume, short retention	Daily dump, 30-day retention
Active production, multi-day retention enabled (`KNEO_SERV_RETENTION_*`)	Hourly dump, 7-day retention; daily off-site copy
Audit-heavy compliance workloads	Per-hour dump kept for the compliance window; verified test-restore monthly

The relevant env vars are in `environment.md § Retention`. A retention policy that prunes runs after 7 days needs backups newer than 7 days, or the restore set is empty.

Verifying a restore

Backups are unproven until they have been restored. Verify on the schedule below, not after a real incident.

1. Provision a scratch host or namespace and restore the backup into it.
2. Start `kneo-serv` against the restored database.
3. Verify dependencies: `bash curl -sf http://127.0.0.1:8000/readyz | jq '.metadata.ready' # → true`
4. Verify a known run survived: `bash curl -sf "http://127.0.0.1:8000/v1/runs?limit=5" \ -H "Authorization: Bearer $OP_TOKEN" | jq '.runs[].run_id'`
5. Run the deployment smoke against the restored stack (`deployment_smoke.md`). It exercises run create → fetch → cancel and confirms checkpoints persist.
6. Verify audit events from before the backup are present: `bash curl -sf "http://127.0.0.1:8000/v1/audit-events?limit=5" \ -H "Authorization: Bearer $OP_TOKEN" | jq '.events[].event_type'`

Recommended cadence: monthly restore drill into a scratch environment, plus a restore drill immediately before any major upgrade.

Rolling back after a failed upgrade

Migrations are schema-forward and not safe to downgrade in place. If a new release misbehaves and the issue can't be patched forward:

1. **Stop the service.** Quiesce writers; the proxy can keep returning `503` from `/readyz` until step 5. `bash docker compose --env-file deploy/production.env stop api`

2. **Restore persistence** from the pre-upgrade backup using the PostgreSQL or SQLite procedure above.
3. **Re-install the previous version.** Pin the image tag or reinstall the pip package at the prior version. Update Compose / Kubernetes manifests accordingly.
4. **Restart the service.** `bash docker compose --env-file deploy/production.env start api`
5. **Verify** with `curl /readyz` and the deployment smoke (`deployment_smoke.md`).

Keep the pre-upgrade backup until you have verified the new version through at least one business cycle.

Disaster recovery checklist

Scenario	Recovery
Lost the host, database survived	Provision new host → install `kneo-serv` → point `KNEO_SERV_DATABASE_URL` at the surviving DB → start.
Lost the database	Provision DB → restore latest dump → start service → verify `/readyz` and a known run.
Lost host and database	Provision DB → restore latest off-site dump → provision host → start service → verify.
Corrupted checkpoints for one run	Use `GET /v1/runs/{run_id}/checkpoints/diff` to identify the bad checkpoint; cancel and re-run from the last good step. The DB itself is fine.
Restore brought back stale data, signs of mismatch	See `troubleshooting.md § 2.5` for the recovery shape.

What this page does not cover

- **Performance and capacity sizing.** Covered in its own guide: `performance.md` — throughput, latency, store choice, and the bench harness for reproducing numbers on your own hardware.
- **The Python backup API surface.** Stays in `service_api.md § Backup and restore` .
- **Release-team verification gates** for the GA cut. Those live in `release_checklist.md` .

Upgrade guide

Source: `docs/user/upgrade.md`

Conventions for upgrading Kneo Agent Platform (`kneo-serv`) between releases, plus version-specific notes when a release has breaking changes.

For the release process itself (gates, tagging, artifacts), see release_checklist.md. For the supported `kneo_agent` SDK range, see sdk_alignment.md.

Versioning

Kneo Agent Platform follows semantic versioning:

- **Patch** (`0.1.0` → `0.1.1`): bug fixes; persistence schemas, route contracts, CLI commands, and env-var names do not change.
- **Minor** (`0.1.x` → `0.2.0`): additive changes. Persistence schemas may add new tables or columns with migrations; routes and CLI may add new surfaces. Existing surfaces remain available with the same shape unless the release notes call out an exception.
- **Major** (`1.x` → `2.0`): may remove or change surfaces. Read the release notes before upgrading; expect to update calling code.

As of `1.0.0` (GA), the `/v1` HTTP API and the `kneo` CLI are stable contracts under semantic versioning: additive changes within the major, deprecation windows before any removal, and no silent `/v1` breaks. (Pre-`1.0`, behavior corrections could land in a minor under the fixes-vs-breaks policy; that latitude is retired.)

The HTTP API is also versioned at the URL prefix (`/v1`); legacy unversioned routes remain available alongside `/v1`. See design.md § 13 and contract_stability.md.

Standard upgrade procedure

1. **Read the release notes** for every minor/major version between your current and target version. Patch upgrades only need the latest patch's notes. The reading-order index (newest first) is releases/README.md; the current release's notes are release_notes_1.0.0.md.
2. **Pin the target version** in your dependency manifest: `kneo-serv[deploy]==X.Y.Z`
3. **Stop traffic** to the service (or drain via a load balancer). Background runs that are queued will be reclaimed by the worker after restart; in-flight runs that complete during the drain will record normally.
4. **Back up persistence.** Follow `backup_and_recovery.md` (`pg_dump` for PostgreSQL, `backup_sqlite_database()` for SQLite). Keep the backup until you have verified the new version through at least one business cycle.

5. **Install the new version** in your deployment image or environment.
6. **Restart the service.** Migrations apply automatically at startup. Watch the structured log for `migration` events and any `migration_failed` errors.
7. **Verify** with `GET /readyz` and the deployment smoke script: `bash python scripts/deployment_smoke.py --base-url http://<host>:<port>`
8. **Resume traffic.**

If `GET /readyz` does not return 200 within a few seconds of restart, see troubleshooting.md § 1.2.

Persistence migrations

Every store that has a schema (`SQLiteRunStateStore`, `PostgresRunStateStore`) tracks its schema version and applies forward-only migrations on first connection. Migrations are idempotent and never drop columns or rows on their own. The file-based stores have no schema; they tolerate older record shapes through the row decoder.

If a migration fails, the service refuses to serve requests rather than running on a partially-migrated schema. Fix the underlying cause (usually a permissions or disk-space problem), then restart.

Downgrades are not supported. Restore from backup if you need to revert.

For contributors authoring new migrations (conventions, the dialect portability rules, the test patterns), see `docs/dev/migrations.md`.

Spec migrations

The YAML spec format is versioned at `version: v1`. The compiler accepts older shapes through automatic normalization, but for clarity the CLI can write upgraded specs to disk:

```
kneo spec migrate legacy_agent.yaml --output migrated_agent.yaml
kneo spec migrate migrated_agent.yaml --check --json
```

Specs that pass `kneo spec validate` on the source version will continue to compile after upgrading; specs that hit deprecation warnings should be migrated proactively before a future release removes the fallback.

Signed bundles created with `kneo spec bundle sign` are tied to the signing key, not the kneo-serv version, so bundles signed before an upgrade continue to verify after as long as the signing key is unchanged.

SDK compatibility

`kneo-serv` declares a `kneo-agent` range in `pyproject.toml`. When upgrading kneo-serv, let pip resolve the matching SDK; do not pin SDK versions outside that range. The compatibility tests (`tests/test_sdk_compatibility.py`) assert the SDK surface used by the service, so a version mismatch surfaces as a test failure.

If you maintain custom runtimes or middlewares that import directly from `kneo_agent`, run those compatibility tests after upgrading and update imports in lockstep.

Configuration changes

Environment-variable names and defaults are part of the public surface. Changes are recorded in environment.md and called out in release notes:

- New variables default to behavior consistent with the previous release.
- Renamed variables retain a deprecation alias for at least one minor release; a startup warning is emitted when the alias is used.
- Removed variables are removed only at major versions.

After upgrading, diff your env file against the latest deploy/production.env.example (or `staging.env.example`) to spot any new optional variables.

CLI changes

The `kneo` CLI is regenerated each release; see cli_reference.md for the current shape. New subcommands are additive within minor releases. Subcommand behavior may change at major releases — check the release notes.

CLI profiles stored at `~/.kneo_serv/profiles.json` carry forward across releases. The profile schema is itself versioned and migrated in place.

Version-specific notes

This section grows as releases ship. Each entry should describe what changed, what action operators must take, and how to verify the upgrade.

0.1.0 — initial release

No upgrade applies; this is the first published version. See release_notes_0.1.0.md for scope, capabilities, and verified release-candidate steps.

0.2.0 — first public distribution

This is the first cut to publish a real `kneo-serv` package. 0.1.0 and 0.1.1 shipped as GitHub Release artifacts only; 0.2.0 is the first version available via `pip install kneo-serv` and `docker pull ghcr.io/kneo-agent/kneo-serv`.

Version trajectory on PyPI: `0.0.0 → 0.2.0`. The `kneo-serv 0.0.0` placeholder published on 2026-05-14 reserved the distribution name; it shipped an empty importable module with no `kneo` CLI binary (no `[project.scripts]` entry). Any user who tried `pip install kneo-serv && kneo --version` during the placeholder window saw `kneo: command not found` — 0.2.0 is the first cut to install the binary. The placeholder is yanked once 0.2.0 ships; existing explicit `==0.0.0` pins still resolve, but default `pip install kneo-serv` jumps straight to 0.2.0.

Install paths: - `pip install kneo-serv` — first time this works end-to-end. - `docker pull ghcr.io/kneo-agent/kneo-serv:0.2.0` (and `:0.2`, and `:latest`) — first time the image is available without a local build.

Deployment migration for operators on 0.1.x using `compose.yaml` with the bundled `build: context: .`: - Default flow becomes `docker compose pull && docker compose up -d` against the GHCR image. - The `build:` block stays in `compose.yaml` for contributors and the CI smoke test (`docker compose up --build`). - No required changes to `deploy/production.env` or `deploy/staging.env` from 0.1.1.

Persistence schemas: unchanged from 0.1.1. No migrations required.

Feature additions visible to operators (full per-feature detail in `release_notes_0.2.0.md`): - `kneo spec lint` — CI-friendly validator subcommand that exits non-zero on any warnings or errors. - Retention windows now live in `.kneo/config.yaml` under a `retention:` block, with env vars as the operator override. - Human-task expiration via `PlatformManager.prune_expired_human_tasks()` — paused runs whose human-step deadline has passed transition to a new `expired` status and emit `human.expired` audit events. - Two new reference example specs: `concurrent_review_workflow.yaml` and `group_chat_workflow.yaml`. - Docker-based local PostgreSQL integration testing via `python scripts/postgres_test.py`.

No breaking changes to spec syntax, HTTP API contracts, CLI command names, env-var names, or persistence schemas. Specs that validated under 0.1.1 continue to validate under 0.2.0.

0.2.1 — `/healthz` version and Docker `/app` permission fix

Patch release fixing two regressions discovered while smoke-testing the published 0.2.0 image. Both are bug fixes; no new features, no contract changes.

Upgrade: - `pip install -U kneo-serv` (resolves to `0.2.1`). - `docker pull ghcr.io/kneo-agent/kneo-serv:0.2.1` — `:0.2` and `:latest` now resolve to the 0.2.1 digest.

What was broken in 0.2.0: - `GET /healthz` returned `"version":"0.1.0"` from the 0.2.0 image because `HealthResponse.version` was a hardcoded string literal. 0.2.1 resolves the field

dynamically via `importlib.metadata.version("kneo-serv")`. - Plain `docker run -p 8000:8000 ghcr.io/kneo-agent/kneo-serv:0.2.0` crashed on startup with `PermissionError: [Errno 13] Permission denied: '.kneo'` because `/app` was root-owned but the container drops to the non-root `kneo` user before creating the SQLite-fallback path. 0.2.1 adds `chown -R kneo:kneo /app` to the install layer. The Docker Compose deployment path was unaffected (it pins `KNEO_SERV_DATABASE_URL` to PostgreSQL).

Persistence schemas: unchanged from 0.2.0. No migrations required.

No breaking changes to spec syntax, HTTP API contracts, CLI command names, env-var names, or persistence schemas.

0.2.2 — FastAPI `info.version` fix + post-0.2.0 docs sweep

Patch release fixing one regression in the same family as 0.2.1 plus a documentation sweep. No feature changes, no contract changes, no schema changes.

Upgrade: - `pip install -U kneo-serv` (resolves to `0.2.2`). - `docker pull ghcr.io/kneo-agent/kneo-serv:0.2.2` — `:0.2` and `:latest` now resolve to the 0.2.2 digest.

What was broken in 0.2.1: - `GET /openapi.json` returned `info.version: "0.1.0"` from the 0.2.1 image because the FastAPI app constructor in `kneo_serv/service/app.py` still pinned a hardcoded literal. The 0.2.1 cut fixed `HealthResponse.version` but missed this parallel occurrence. 0.2.2 resolves both via the same `importlib.metadata.version("kneo-serv")` helper, called at app-construction time.

Documentation: - Forward-looking plan docs and "as of 0.1.0" framing in user/dev docs swept to match the 0.2.x shipped reality. No content lost — historical files (CHANGELOG entries, shipped release notes, the archived `0.2.0` tracker, ADRs) are unchanged.

Persistence schemas: unchanged from 0.2.1. No migrations required.

0.3.0

Next additive minor on the 0.2.x line. No breaking changes to spec syntax, HTTP API contracts, CLI command names, env-var names, or persistence schemas. Full narrative in `release_notes_0.3.0.md`.

Upgrade: - `pip install -U kneo-serv` (resolves to `0.3.0`). - `docker pull ghcr.io/kneo-agent/kneo-serv:0.3.0` — `:0.3` and `:latest` now resolve to the 0.3.0 digest. The image is now signed (cosign keyless via Sigstore) and ships with a CycloneDX SBOM attestation; verification commands are in `supply_chain_review.md § Verification commands`.

SDK floor bump: - The `kneo-agent` SDK floor moves from `>=1.1.1` to `>=1.2.0`. Pip auto-resolves on `pip install -U kneo-serv`, but operators pinning the SDK separately (e.g. via a constraints file

or a monorepo lockfile) must ensure their install is on `1.2.0` or newer. The compat test suite passed against `kneo-agent 1.2.0` throughout the 0.2.x line; the floor was kept low to avoid forcing 0.1.x users to upgrade. 0.3.0 is the natural inflection point to lift it.

New `timed_out` lifecycle status: - Runs that hit their run-level deadline transition to a new terminal `timed_out` status (alongside `completed`, `failed`, `cancelled`, `expired`). Operator tooling that switches on `state.status` should accept it as terminal — e.g. dashboards, alerting rules, retention sweeps (which the platform's own `RetentionPolicy.run_statuses` already includes). - The `error.type` field on a timed-out run is `run_timed_out`, distinct from `human_task_expired` (which the existing `expired` status uses).

New runtime surfaces: - `start_run_from_spec(..., timeout_seconds=N)` and `run_from_spec(..., timeout_seconds=N)` accept an optional wall-clock deadline. Operator-callable `PlatformManager.prune_timed_out_runs()` walks runs and force-cancels those past their deadline. Same operator-cron pattern as `prune_retention()` and `prune_expired_human_tasks()` — no built-in scheduler. - The human-task `on_timeout: continue` and `on_timeout: escalate` literals are now wired in the runtime (they were accepted by the spec but silently treated as `fail` in 0.2.x). Operators with specs that declared these literals will see the documented behaviour for the first time. Audit consumers should expect new event types: `human.continued`, `human.continue_failed`, `human.escalated`, `run.timed_out`. - New route `GET /v1/runs/{run_id}/policy-report` returns the spec policy report for a stored run, no spec bundle required client-side. Auth: `specs:read` scope (same as the existing `POST /v1/specs/policy-report`).

New observability surfaces: - Three new platform-side OpenTelemetry spans (`kneo.queue.dispatch`, `kneo.worker.lease`, `kneo.continuation.lock`) join the SDK's agent-boundary spans when `KNEO_SERV_OTEL_ENABLED=true`. Pre-existing OTel pipelines pick them up automatically once telemetry is enabled — no extra configuration required. See `observability.md § Platform-side spans`.

Persistence schemas: unchanged. The new `RunState.deadline_at` and `Checkpoint.iteration` fields default to `None` and `1` respectively in the dataclass, so existing rows round-trip cleanly through the JSON-payload SQLite / PostgreSQL stores.

0.4.0

Next additive minor on the 0.3.x line. **No breaking changes** to spec syntax, HTTP API contracts, CLI command names, env-var names, or persistence schemas. Specs that validated under 0.3.x continue to validate under 0.4.0. The cut is a **docs + tooling release** — runtime semantics are identical to 0.3.0. Full narrative in `release_notes_0.4.0.md`.

Upgrade: - `pip install -U kneo-serv` (resolves to `0.4.0`). - `docker pull ghcr.io/kneo-agent/kneo-serv:0.4.0` — `:0.4` and `:latest` now resolve to the 0.4.0 digest. Image continues to be signed (cosign keyless via Sigstore) and ships with a CycloneDX SBOM attestation; the 0.4.0 cut

adds a Trivy CVE scan report attached to the GitHub Release. Verification commands are in `supply_chain_review.md § Verification commands`.

SDK floor: unchanged. The `kneo-agent` floor stays at `>=1.2.0` — same as 0.3.0. No operator action required for operators pinning the SDK separately.

New auto-generated API reference: the docs site at `kneo-agent.github.io/kneo-serv/` gains a new top-level **API Reference** nav section with 17 pages (16 subpackages + `sdk`), rendered at build time by `mkdocstrings` from the Python docstrings. Operator surface unchanged — the API ref is a **developer lookup surface**, not a runtime change. See `docs/api/README.md` for the index.

Image vulnerability scanning (Trivy): the release pipeline now scans the pushed GHCR image with Trivy under the CVSS≥7 policy (HIGH/CRITICAL findings block the publish step). On every release-tag build, the JSON scan report is attached to the GitHub Release as the `trivy-report-<version>` artifact, 90-day retention. Deployers can re-run the scan locally with `trivy image ghcr.io/kneo-agent/kneo-serv:<tag>`; full policy + escape hatch documented in `security_hardening.md § Image vulnerability scanning`.

Developer-facing changes (no operator surface impact): - Ratcheting ruff D-rule gate (`D100` / `D101` / `D102`) now enforced project-wide for `kneo_serv/**/*.py`. New public classes / methods without docstrings fail CI. Forks adding code should follow the Google docstring convention; the chain-reference files are `security/secrets.py` and `platform/manager.py`. - Full mypy strict coverage across `kneo_serv/`. The `[[tool.mypy.overrides]]` block in `pyproject.toml` now covers every public module. Forks that subclass or extend public types should expect `disallow_untyped_defs` + `warn_return_any` + `strict_equality`. - `mkdocstrings[python]>=0.27` added to the `docs` optional-dep block. Operators using `pip install kneo-serv` (without `[docs]`) are unaffected — the dep is build-time only for the rendered site.

New 0.3.0-feature worked examples: - `examples.md` picked up a *Timeout branches* subsection on the `human_approval_workflow.yaml` entry covering the `on_timeout: fail/continue/escalate` literals (all wired since 0.3.0). - New `examples/run_with_timeout.py` walks through `start_run_from_spec(..., timeout_seconds=N)` + `prune_timed_out_runs()`. Companion to the human-task timeout example above.

Persistence schemas: unchanged. No new fields, no migrations.

0.5.0

Next additive minor on the 0.4.x line. **No breaking changes** to spec syntax, HTTP API contracts, CLI command names, env-var names, or persistence schemas. Specs that validated under 0.4.x continue to validate under 0.5.0.

Upgrade: - `pip install -U kneo-serv` (resolves to `0.5.0`). - `docker pull ghcr.io/kneo-agent/kneo-serv:0.5.0` — `:0.5` and `:latest` resolve to the 0.5.0 digest; image signing, SBOM

attestation, and the Trivy scan gate are unchanged from 0.4.0.

SDK floor: unchanged. The `kneo-agent` floor stays at `>=1.2.0`.

Bug fix — checkpoint-callback metadata now survives on the final run record: in 0.4.x and earlier, `execute_run` and `continue_run` saved a stale in-memory `RunState` at the end of a run, overwriting the `step_iteration_counts` and `execution_context` that the checkpoint callback had written to `RunState.metadata` during the run. The values survived on the *checkpoints* but were missing from the *run record*. 0.5.0 re-reads the persisted run before the final save. **Operator impact**: if you read per-step iteration metadata off the final `RunState` (via `GET /v1/runs/{id}` metadata or the `PlatformManager` API) and worked around its absence, that metadata is now present. Checkpoint contents are unchanged. No migration or data backfill — the fix only affects runs executed under 0.5.0 onward.

New performance and capacity guide: `performance.md` documents throughput, latency, the SQLite-vs-PostgreSQL capacity trade-off, and the tuning knobs, with a reproducible bench harness (`scripts/bench`). No runtime change — guidance and tooling only.

Single-team on-prem operability (all additive; defaults preserve 0.4.x behaviour): - **Worker concurrency.** `KNEO_SERV_WORKER_CONCURRENCY` (default `1`) runs a pool of in-process worker threads; `KNEO_SERV_WORKER_LEASE_SECONDS` (default `300`) sets the queue lease. Default `1` reproduces the prior single-worker behaviour. Sizing guidance in performance.md. - **Prometheus `/metrics`.** New unauthenticated `GET /metrics` (root path only), opt-out via `KNEO_SERV_METRICS_ENABLED=false`. **Operator action**: restrict it to your monitoring network (reverse proxy or the env flag). See observability.md § Prometheus `/metrics` . - **Overload backpressure.** `KNEO_SERV_MAX_QUEUE_DEPTH` (default `0` = unlimited, i.e. unchanged). When set, `POST /v1/runs` (async) returns `503` with `Retry-After: 5` once the queue is full — make async callers retry on `503`. - **Poison-run dead-letter.** `KNEO_SERV_QUEUE_MAX_ATTEMPTS` (default `5`) fails a run re-leased past the cap with a `dead_letter` error + `run.dead_lettered` audit event. **Behaviour change**: before 0.5.0 a run that repeatedly crashed its worker was re-leased indefinitely; it is now dead-lettered after 5 attempts. Set `0` to restore unbounded retries. - **Graceful drain.** On SIGTERM the worker pool finishes in-flight runs and stops claiming new ones, so container rollouts no longer interrupt a run.

Persistence schemas: unchanged. No new fields, no migrations.

0.6.0

Multi-process PostgreSQL hardening + uptake of the Kneo Agent SDK 2.x line. **Additive persistence migrations only** (a new checkpoint uniqueness index); no breaking spec-syntax, CLI, or env-var-removal changes. Two intentional *observable* changes (error codes, `RunConfig` defaults) are called out below.

Upgrade: - `pip install -U kneo-serv` (resolves to `0.6.0`). - `docker pull ghcr.io/kneo-agent/kneo-serv:0.6.0` — `:0.6` and `:latest` resolve to the 0.6.0 digest; image signing, SBOM

attestation, and the Trivy scan gate are unchanged.

SDK floor bumped to `kneo-agent>=2.2.0,<3.0.0` (from `>=1.2.0,<2.0.0`). The service installs this for you; if you pin `kneo-agent` yourself, move your pin onto the 2.x line. The bump picks up SDK security fixes (MCP cross-origin redirect refusal + URL-scheme validation; broader secret redaction). Two SDK 2.x behaviour deltas a deployer may notice: - **The SDK no longer auto-retries tool calls.** kneo-serv's own retry knobs (`KNEO_SERV_PROVIDER_RETRIES`, `KNEO_SERV_MCP_RETRIES`, …) are unchanged and still apply; only the SDK's internal per-tool retry default flipped off. - **Spec/agent `with_defaults` for `temperature` / `max_iterations` now actually apply** — see the `RunConfig` change below.

Observable change — error-code remap (action may be required). The public `error` field in error responses is now a stable, snake_case code decoupled from internal Python class names:

Status	Old `error`	New `error`
404	`KeyError`	`not_found`
400	`ValueError` / `FileNotFoundError`	`invalid_request`
500	`<ExceptionClassName>` (+ raw message)	`internal_error` (generic message)

`queue_full`, `resource_locked`, `unauthorized`, `forbidden`, and the idempotency codes are unchanged. If a client matched on the old class-name codes (`KeyError`, `ValueError`, …), update it to the new codes. The error envelope — `{"detail": {"error": "...", "message": "..."}}` — is otherwise unchanged and is now published in the OpenAPI schema as `ErrorResponse` / `ErrorDetail`. 500 responses no longer echo the exception message, and `/readyz` probe failures no longer leak the underlying error detail (both are logged server-side instead).

Observable change — `RunConfig` defaults now merge. Before 0.6.0, a run that didn't specify `max_iterations` / `temperature` had them force-set to `10` / `0.7`, silently overriding a spec author's `with_defaults(...)`. 0.6.0 leaves them unspecified so the agent/skill defaults apply (SDK 2.x merge semantics). If a spec set `temperature: 0.2` (or a custom `max_iterations`) and you relied on the run *ignoring* it, the spec value now takes effect. To force a value regardless of the spec, set it explicitly on the run config. A malformed `temperature` (non-numeric / `bool` / `NaN` / `inf`) now returns `400 invalid_request` instead of a 500.

New operator knobs (all additive; defaults preserve 0.5.x behaviour): - **`KNEO_SERV_RETENTION_AUDIT_DAYS`** (and project-config `retention.audit_days`) — prune audit events older than N days. The audit table is otherwise unbounded; set this on long-lived deployments. - **`KNEO_SERV_SHUTDOWN_TIMEOUT_SECONDS`** (default `30`) — how long SIGTERM shutdown waits for in-flight runs to finish. A run still executing past the timeout is interrupted by process exit but stays claimed

and is re-leased / retried (not lost); set this **and** your orchestrator's termination grace period ≥ your longest run step to drain without a restart. - **Token-usage metrics.** `/metrics` now exposes `kneo_tokens_input_total`, `kneo_tokens_output_total`, and `kneo_tokens_total` counters; usage is also on the run record and `run.created` audit metadata when the runtime reports it. - **Idempotency in-progress.** A duplicate `POST` arriving while the first same-key request is still in flight now returns `409 idempotency_key_in_progress` (previously the two could race). Treat `409` as "retry shortly".

Persistence schemas: one **additive** migration — a `UNIQUE(run_id, sequence)` index on `checkpoints` (migration v3), which de-duplicates any pre-existing duplicate `(run_id, sequence)` rows on first start. PostgreSQL queue/lease timestamp columns are widened `REAL → DOUBLE PRECISION` in place (a precision fix). No data backfill or operator action required; both apply automatically on the first start under 0.6.0.

0.7.0

Finishing the 0.6.0 lease-liveness story plus an on-prem operability cluster. **No breaking changes, no persistence migration, no SDK-floor change** — the schema version is unchanged from 0.6.0 and the SDK floor stays `kneo-agent>=2.2.0,<3.0.0`. Two *behaviour* notes below are worth reading before you upgrade; everything else is additive and default-off / default-unset.

Upgrade: - `pip install -U kneo-serv` (resolves to `0.7.0`). - `docker pull ghcr.io/kneo-agent/kneo-serv:0.7.0` — `:0.7` and `:latest` resolve to the 0.7.0 digest. The release pipeline now runs an **in-pipeline cosign verify self-check** (`cosign verify` + `verify-attestation` against the pushed digest); image signing, SBOM attestation, and the Trivy gate are otherwise unchanged.

Behaviour note — `worker_lease_seconds` is now a liveness window, not a run-time cap. A worker now renews its queue lease for the life of a run (a heartbeat renewing at ~`worker_lease_seconds / 3`), so a healthy long run never lets its lease lapse and get reclaimed mid-flight. The lease therefore no longer bounds how long a run may take — it bounds how long a *crashed* worker's run stays unreclaimable. If you raised `worker_lease_seconds` in 0.5.x/0.6.x to "fit" your longest run, you can lower it back toward your crash-detection latency. No action is required; the default is unchanged and shorter leases are now safe.

Behaviour note — the per-run token ceiling is a post-run boundary check, not a mid-flight kill. A run can be capped at a maximum input+output token budget via the SDK's `TokenBudgetMiddleware`, configured per agent with `model.token_budget` in the spec (a **positive** integer — a non-positive value is rejected at spec validation, not silently ignored) or deployment-wide with `KNEO_SERV_TOKEN_BUDGET` (the spec field wins). The middleware checks reported usage **after** each run and then raises `TokenBudgetExceeded`, surfaced as `400 token_budget_exceeded`. Consequences: - A run that overshoots within a single step finishes that step before failing — size the ceiling as a spend **backstop**, not a precise hard stop. - `on_missing="ignore"`: a runtime that doesn't report `metadata["usage"]` never spuriously fails the ceiling. Unset (the default) means no ceiling.

New operator knobs (all additive; defaults preserve 0.6.x behaviour): - **`KNEO_SERV_AUDIT_EXPORT_ENABLED`** — when set, every persisted (already redacted) audit event is also emitted as a JSON line on the dedicated `kneo_serv.audit` logger, from the single `record_audit_event` chokepoint. Attach a logging handler to forward to a file / syslog / SIEM. Off by default; export failures never break the run path. - **`KNEO_SERV_TOKEN_BUDGET`** — deployment-wide per-run token ceiling (see the behaviour note above). A spec's `model.token_budget` overrides it. - **Local / self-hosted LLM endpoints.** The native (`openai`) runtime now reads `model.extra.base_url` and an API key — `model.extra.api_key_ref` resolved through the `SecretResolver` , or a literal `api_key` escape hatch — and threads them into the OpenAI-compatible client, so a spec can target Ollama / vLLM / llama.cpp / LocalAI. Unset fields preserve the hosted-OpenAI default; a literal `api_key` is redacted from audit / list surfaces.

New spec fields (optional, additive — old specs are unaffected): - **Human-request taxonomy.** `components.humans.*` accepts `request_type` (`approval` / `review` / `correction` / `selection` / `freeform`), `options` , `default_option` , `context` , and `response_role` . `validate_semantics` rejects a `default_option` outside `options` and a `selection` without `options` . When `response_role` is set, the reviewer's reply folds into the resumed run's message thread with that role. `GET /v1/human-tasks/{id}` now also returns the paused run's **redacted `messages` thread** alongside the pending `request` (same auth scope; no new route). A client that only reads the existing `request` field is unaffected.

Persistence schemas: **no migration.** The store schema version is unchanged from 0.6.0; `RunStateStore` gains a `schema_version` / `close` Protocol surface (behaviourally a no-op on the schema-less stores), but no on-disk change applies on upgrade.

0.9.0

Reliability & retention. **No breaking changes, no persistence migration, no SDK-floor change** — the spec schema version is unchanged and the SDK floor stays `kneo-agent>=2.2.0,<3.0.0` . Persistence additions (the idempotency prune, count queries) are additive-only; rollback to 0.8.0 is safe (no persisted-field removals).

Behavior corrections to review before upgrading. Each corrects shipped behavior that contradicted its own documented/validated contract (the fixes-vs-breaks test in the new contract-stability policy, adopted this cut). If you built automation against the old behavior, adjust:

- **Handoff `round_robin` runs report `completed`** after a full rotation (previously every successful rotation persisted as `failed` / `max_iterations`). Alerts keyed on that false failure will go quiet.
- **`on_error: continue` / `fallback` execute.** Workflows that declared error tolerance but relied on the hard failure will now proceed: `continue` passes the step's input through; `fallback` runs the referenced step. See the run lifecycle guide.
- **List `total` is the true store count** — it previously capped silently at the 10 000-row fetch window. Dashboards asserting `total ≤ 10000` should read the pagination block. Run list

items now carry `trace_event_count` instead of each run's full `trace_events` array (the trace lives at `GET /runs/{id}/trace`).
- **Token-usage metrics survive redaction** (`input_tokens` etc. were `[REDACTED]` everywhere). Cost dashboards start receiving real values.
- **Resume/continue are fenced**: resuming a run that is not `blocked`, or continuing a terminal-but-not-failed / live-leased run, returns **`409 run_state_conflict`** instead of silently re-executing. Cancelling a blocked run removes its task from `GET /human-tasks`.
- **A per-attempt timeout no longer retries by default** — the abandoned attempt may still be running, so the retry double-executed non-idempotent calls. This applies to provider calls AND to workflow steps/nodes that set `timeout_seconds` + `max_retries`. Opt back in per surface: `KNEO_SERV_PROVIDER_RETRY_ON_TIMEOUT=true` (or `retry_on_timeout` in spec retry config) for providers, `KNEO_SERV_WORKFLOW_RETRY_ON_TIMEOUT=true` for workflow steps/nodes. MCP *connect* timeouts are the exception: they cancel the connect coroutine cleanly, so configured MCP retries do retry them.
- **Stricter validation** (pure checks; previously these crashed at runtime): graph `kind: human` nodes (`E_GRAPH_NODE_HUMAN_UNSUPPORTED`), memory blocks without `policy` (`E_MEMORY_POLICY_REQUIRED`), guardrail items missing `id`/`type` (`E_GUARDRAIL_FIELDS`). Invalid path-based specs now return `200 {valid: false}` from `/specs/validate` (was 400).
- **Environment policies set via REST are enforced** on run/compile when the request names the environment — a deployment blocked by policy returns `403 environment_policy_blocked`. Verify your stored policies say what you mean before upgrading production.
- **Stricter env-var parsing**: invalid numeric values in `KNEO_SERV_WORKER_*` / queue knobs now fail startup instead of silently running defaults; same for the new strictly-parsed knobs.

New knobs (all optional; see environment.md): `KNEO_SERV_RETENTION_IDEMPOTENCY_DAYS`, `KNEO_SERV_RETENTION_RUN_STATUSES`, `KNEO_SERV_TRACE_MAX_EVENTS`, `KNEO_SERV_MCP_CONNECT_TIMEOUT_SECONDS`, `KNEO_SERV_IDEMPOTENCY_LOCK_TTL_SECONDS`, `KNEO_SERV_PROVIDER_RETRY_ON_TIMEOUT`, `KNEO_SERV_WORKFLOW_RETRY_ON_TIMEOUT`, `KNEO_SERV_ARTIFACT_PATH` / `KNEO_SERV_LOG_PATH`.

0.8.0

Declarative spec parity along the tools / MCP / skills axis. **No breaking changes, no persistence migration, no SDK-floor change** — the schema version is unchanged and the SDK floor stays `kneo-agent>=2.2.0,<3.0.0`. One *behaviour* note below is worth reading before you upgrade; everything else is additive and default-unset.

Upgrade: - `pip install -U kneo-serv` (resolves to `0.8.0`). - `docker pull ghcr.io/kneo-agent/kneo-serv:0.8.0` — `:0.8` and `:latest` resolve to the 0.8.0 digest. Image signing, SBOM attestation, and the Trivy gate are unchanged from 0.7.0.

Behaviour note — `overlays` is no longer silently ignored. `POST /v1/runs` (sync + async) and the `/v1/specs/run` / `/compile` / `/validate` / `/policy-report` routes accepted an `overlays` list but dropped it without applying it. From 0.8.0 the overlays are threaded through compile/run, persisted in run metadata, and replayed on resume. If any stored client request or automation passes `overlays`, audit it before upgrading — those overlays now actually change the compiled spec. `overrides` / `strict` are likewise now honored on the `/specs/*` routes that previously dropped them.

Trust note — `spec_path` and `overlays` are filesystem-trusted inputs. Both name paths the *server* reads at compile time. Grant `runs:write` / `specs:read`-scoped keys to callers you trust with that read surface, and see security_hardening.md for the posture before exposing these fields to semi-trusted callers.

New spec surface (all optional, additive — old specs are unaffected):

- **Declarative MCP transports.** A top-level `mcp_servers` block (`transport: stdio | http | sse`, with `command` / `args` / `env` / `cwd` or `url` / `sse_url` / `message_url` / `headers` / `timeout`, plus `max_response_bytes` / `sse_read_timeout` knobs and `verify` / `ca_bundle` / `client_cert` / `client_key` TLS fields) and a `tool.mcp = {server: <name>, name?: <remote_tool>}` reference. Construction happens at build time; the connection is lazy on first tool call, so the spec compiles offline. **Prefer `client_key_ref`** (resolved via the `SecretResolver`) over inline `client_key` — the inline spec is persisted unredacted into run metadata, and the TLS field names are redaction terms only on audit/list surfaces. `verify: false` draws a validation warning.
- **Agent-as-tool.** `tool.agent: <name>` backs a tool with another declared agent. A tool must be backed by exactly one of `implementation` / `mcp` / `agent` — a tool with none is now a validation error (`E_TOOL_NO_BACKING`) instead of being silently dropped at build.
- **Workflow-as-agent.** `agent.as_agent: <workflow>` backs an agent with a declared workflow; only `name` / `description` / `system_prompt` are legal alongside it. Cyclic or dangling references across all of these fail at `/specs/validate` (`E_BUILD_CYCLE` etc.), not at runtime.

New API surface (additive):

- **`GET /v1/skills`** — read-only catalog of declared + default discoverable skills; `specs:read` scope, standard pagination, no side effects.
- **`RunCreateRequest.skills`** — per-request `{add, disable}` skills overlay. `add` only enables skills already declared in the spec; out-of-scope overlays are rejected; every overlay is audited (`run.skills_overlay`) and preserved across resume.
- **`GET /v1/human-tasks?status=pending|escalated`** — a real filter now; an unknown value returns 422 where it was previously a silent no-op.
- **`POST /runs/{id}/continue`** accepts an `Idempotency-Key` and replays the stored response on retry; concurrent `/continue` calls are serialized under a per-run lock. `POST`

`/v1/specs/run` now holds the same idempotency lock as `/runs` (409 `idempotency_key_in_progress` on contention).

- **Invalid specs on sync `POST /v1/runs` return `400`** with diagnostics where they previously surfaced as an opaque 500. 413 (`payload_too_large`) is now published in the OpenAPI error responses.

Persistence schemas: **no migration.** No new persisted fields; rollback to 0.7.x after running 0.8.0 is persistence-safe (the new request/spec fields are request-scoped or compile-scoped only).

0.10.0

Theme: **performance & capacity / 1.0 runway.** A correctness/security/ hardening cut. Additive-only; **no migration**; SDK floor held at `kneo-agent>=2.2.0,<3.0.0`. 0.10.0 is a normal additive minor — **not** the `1.0` cut.

Intentional behavior changes (each corrects provably-wrong shipped behavior per the contract-stability policy; act if you keyed on the old behavior):

- **`tool`-stage `redact`/`warn` guardrails now actually enforce.** Before 0.10.0 a declared `tool`-stage guardrail validated, satisfied the production `require_guardrails` gate, and deployed — but was never wired into the runtime, so it never ran. `redact`/`warn` now execute in the tool-call chain. **Action:** if a deployment declared a `tool`-stage `redact`/`warn` guardrail, it was *unprotected* until now (disclosed on fix) — re-review it and confirm the now-live behavior is what you want (e.g. a `redact` action will now actually redact tool output).
- **Raising tool-stage guardrail actions are now rejected at `/specs/validate`** (`E_GUARDRAIL_ACTION_UNSUPPORTED`). A tool-stage guardrail with the default `block` (or escalate/human_review/retry/revise) cannot abort the run yet — the SDK bridge executor's per-tool-failure contract converts the raised violation into a recoverable result, so it would fail open. Rather than ship that, such specs now fail validation. **Action:** for tool-stage guardrails use `redact`/`warn`, or move a blocking check to the `input`/`output` stage (those enforce `block` correctly). True tool-stage block-enforcement is planned for 0.11.0. Note: a tool-stage guardrail with **no explicit `action`** defaults to `block`, so add `action: redact` (or `warn`) to such specs.
- **Guardrails with a non-`middleware` `mode` are now rejected at `/specs/validate`** (`E_GUARDRAIL_MODE_UNSUPPORTED`). Only the `middleware` attachment is wired; other modes (`runtime`/`tool`/`workflow`) were silently dropped. **Action:** remove `mode` (it defaults to `middleware`) or set it to `middleware`.
- **`workflow`-stage guardrails are now rejected at `/specs/validate`** (`E_GUARDRAIL_STAGE_UNSUPPORTED`). No runtime hook enforces them yet, so a spec declaring one previously validated green and silently did nothing. **Action:** remove `workflow`-stage guardrail blocks (or move the control to a `tool`/`input`/`output` stage); such specs will now fail validation.

- **A `kind: workflow` step containing a human-approval step is rejected at `/specs/validate`** (`E_STEP_WORKFLOW_NESTED_HUMAN`). Such specs used to validate and then complete the run with the *unapproved* output. **Action:** lift the human-approval step to the top-level workflow (the supported pattern — it blocks and resumes correctly).
- **Secret redaction now covers pluralized credential keys** (`api_keys`, `refresh_tokens`, `KNEO_SERV_API_KEYS`). Single-segment usage counters (`input_tokens`, `max_tokens`) are unaffected. **Action:** none expected; if you scraped a redacted log/trace/audit field expecting a plural credential key to appear in the clear, it no longer will.
- **Release packaging:** the container image's public tags (`:X.Y.Z` / `:X.Y` / `:latest`) and the GitHub Release are now gated behind the Trivy CVE scan and the coverage/postgres lanes (the `release → scan → gated ship` split). No operator action; relevant only if you build the image from this repo's workflow.

Persistence schemas: **no migration.** Terminal-write atomicity, the persistent idle-poll worker, and checkpoint-prune liveness are behavior-internal; no new persisted fields. Rollback to 0.9.x is persistence-safe.

0.11.0

0.11.0 is a **breaking, 1.0-runway cut**: it ships the two held 1.0-register `/v1` contract changes, plus guardrail-enforcement that turns some previously-rejected specs into accepted-and-enforced ones.

Breaking — `/v1` contract:

- **Async run-create returns `202 Accepted`** (was `200`). `POST /runs` / `POST /v1/runs` with `async_mode=true` now returns **202**; synchronous creates (`async_mode=false`) still return **200**. The response *body* is unchanged. **Action:** if your client asserts `status_code == 200` on async create, accept `202` (or treat `2xx` as success); keep polling `GET /runs/{run_id}` exactly as before. Idempotent replays preserve the 202.
- **Unknown query parameters are rejected with `422`.** Any query-string parameter a route does not declare now returns `422 {"error": "unknown_query_parameters", "unknown": [...]}` on the authenticated `/v1` (and root) surface; through 0.10.x they were silently ignored. `/healthz`, `/readyz`, `/metrics` are exempt. **Action:** remove stray/misspelled query params from API calls; a typo that was previously a silent no-op now errors (which is the point — it surfaces the bug). Request bodies already rejected unknown fields, so this only changes query strings.

Behavior — guardrail enforcement (specs rejected at 0.10.0 now validate):

- **Tool-stage** guardrails with a *raising* action (`block`/escalate/etc.) are now **enforced** — a violation aborts the run (sync → `422`, async → `failed`) instead of failing open. **`E_GUARDRAIL_ACTION_UNSUPPORTED` is no longer raised** at `/specs/validate`.
- **`workflow`-stage** guardrails are now **accepted and enforced per step** (each step's output is checked; block aborts, redact/revise rewrite). **`E_GUARDRAIL_STAGE_UNSUPPORTED` is no**

longer raised. Action: if you relied on these being rejected as a lint, note they now run — audit any `tool` / `workflow`-stage guardrail blocks you had declared "for later."
- Guardrails now also apply to **streaming** runs (`Agent.stream`): input guardrails run before the stream; an output `revise` buffers and rewrites the caller-received text (so a stream with an output guardrail yields the revised result as one chunk rather than token-by-token).

Persistence schemas: **no migration** — all changes are API-surface or behavior-internal; no new persisted fields. Rollback to 0.10.x is persistence-safe (but clients depending on the new 202/422 contract must roll back too).

Downstream: `kneo_client` (and anything pinning the `/v1` contract) needs a coordinated uptake for the 202 + reject-unknown-query-params changes — see its `TODO-0.8.0`.

0.12.0

0.12.0 is an **additive, production-ready minor** (the GA candidate). No breaking `/v1` change ships in this cut; the one deliberate break (spec-path confinement default-on) is *staged* here as a deprecation warning and lands at `1.0.0`.

Behavior change — `POST /specs/run` honors `async_mode`:

- Through 0.11.x, `POST /specs/run` silently ignored `async_mode` and always ran the spec inline, returning `200`. It now mirrors `POST /runs`: with `async_mode=true` it dispatches to the worker queue and returns **`202 Accepted`** with the queued `run_id` (poll `GET /runs/{run_id}`); the synchronous default (`async_mode=false`) still returns **`200`**; idempotent replay preserves the original status. **Action:** a client that sent `async_mode=true` to `/specs/run` and relied on getting a completed run back at `200` will now get `202` + a queued id — switch to polling (as `/runs` callers already do). Clients that only used the synchronous default are unaffected.

Deprecation (becomes a default-on break at `1.0.0`) — spec-path confinement:

- `spec_path` and `overlays` are caller-supplied filesystem paths the service reads at compile time. 0.12.0 adds an opt-in `KNEO_SERV_SPEC_ROOT` env var: set it to an allow-listed root and any path resolving outside it (absolute, `..`-traversal, symlink escape) is rejected `422 spec_path_confined`. **While `KNEO_SERV_SPEC_ROOT` is unset, behavior is unchanged except that an absolute path now logs a `DeprecationWarning`.** At `1.0.0` confinement becomes **default-on** and absolute / out-of-root paths are rejected by default. **Action:** set `KNEO_SERV_SPEC_ROOT` to the directory that holds your specs now — this both closes the path-disclosure surface today and adopts the GA behavior ahead of the `1.0.0` flip. (Held 1.0-register change; see `../dev/contract_stability.md`.)

Also in this cut (no action needed): human-approval (`kind=human`) gates now pause + resume in **every** workflow shape (graph, handoff, group-chat, concurrent — previously sequential only); the `kneo spec explain` CLI command; an enforced seeded backup/restore release gate; and internal

correctness/security fixes (overlay path-confinement, a tool-policy fail-open close, file-store retention-race hardening). See the CHANGELOG.

Persistence schemas: **no migration** — additive only; rollback to 0.11.x is persistence-safe. (A run blocked inside a graph/orchestration workflow on 0.12.0 cannot be resumed after a rollback to 0.11.x, which lacks the continuation support — drain in-flight blocked runs before rolling back.)

1.0.0

BREAKING — spec-path confinement is default-on. Through `0.12.x`, `KNEO_SERV_SPEC_ROOT` was opt-in: with it unset, an absolute or out-of-root `spec_path` / `overlays` was accepted and only logged a `DeprecationWarning`. At `1.0.0` the default flips to **reject**: a caller-supplied path that resolves outside the confinement root is refused with `422 spec_path_confined`. When `KNEO_SERV_SPEC_ROOT` is unset, the confinement root is the **process working directory**.

This break now also covers **`skills[].source`** — a declared skill bundle's filesystem path, which through `0.12.x` was read unconfined (the sibling path that bypassed the `spec_path` / `overlays` confinement and left an authenticated arbitrary-file-read oracle open). At `1.0.0` an out-of-root skill source is rejected like any other spec read, and a `..` / `~` traversal in a skill source is rejected at spec validation.

Action. Pick one:

- **Set `KNEO_SERV_SPEC_ROOT`** to the directory that holds your specs, overlays, and skill bundles (the recommended posture — an explicit allow-listed root). Everything you load by `spec_path` / `overlays` / `skills[].source` must resolve inside it.
- **Or keep specs under the service's working directory** and leave `KNEO_SERV_SPEC_ROOT` unset (the working directory is the default root).

If you deploy with out-of-tree spec or skill paths (e.g. absolute paths to a shared bundle directory), move them under the root or add the root to `KNEO_SERV_SPEC_ROOT` **before** upgrading — otherwise those requests begin returning `422 spec_path_confined`. Inline specs (`spec` in the request body) and per-run skill overlays are unaffected; only caller-supplied filesystem paths are confined.

Local CLI is operator-trusted. Spec-path confinement applies to the **service's** reads of caller-supplied paths (the `/v1` surface, run, resume). The local `kneo` CLI reads the operator's own filesystem directly and is **not** confined to `KNEO_SERV_SPEC_ROOT` — a local operator already owns the filesystem, so `kneo spec validate /any/path.yaml` keeps working from any directory. (When the CLI targets a remote service with `--service-url`, it sends the resolved spec inline; the service applies its own confinement.)

`kneo spec validate` now exits `1` on an invalid spec (was exit `0`, with the diagnostics only in the body). It now works as a CI gate, consistent with `kneo spec lint`; `--json` still prints `valid` + diagnostics. Update any pipeline that relied on exit `0` for an invalid spec.

Persistence schemas: no migration — these are request-validation + CLI changes only.

Rolling back

Schema-forward migrations make in-place downgrade unsafe; the only supported rollback path is **restore from the pre-upgrade backup**, then re-install the previous version.

For the full step-by-step procedure — stop, restore, re-install, restart, verify with the deployment smoke — see `backup_and_recovery.md § Rolling back after a failed upgrade`.

Keep the pre-upgrade backup until you have verified the new version through at least one business cycle.

Reporting upgrade issues

Capture the same context listed in troubleshooting.md § What to capture before opening a bug, plus:

- Source version (`pip show kneo-serv` before the upgrade).
- Target version (after the upgrade).
- Migration log lines from the first start on the new version.
- The exact env file or compose `.env` (with secrets redacted).

Incident response

Source: *`docs/user/incident_response.md`*

On-call entry point for `kneo-serv`. This page is the triage tree — "the service looks wrong, where do I look first?" The symptom-by-symptom deep dive lives in `troubleshooting.md` ; this page sends you there.

For backup and rollback procedures, see `backup_and_recovery.md` . For the API definition of the health endpoints, see `service_api.md § Health checks` .

Triage tree

```
    ┌── /healthz returns 200 ──────  service process is alive
    │                               ─ check /readyz next
    │
1.  ├── /healthz times out ────────  process is down or unreachable
    │                               ─ check the container / supervisor
    │                               ─ check the reverse proxy upstream
    │
    └── /healthz 5xx ──────────────   application crashed mid-request
                                     ─ check stderr / container logs
                                     ─ see troubleshooting.md § 1

    ┌── /readyz returns 200 ──────  dependencies healthy
    │                               ─ problem is in a specific run/spec
    │                               ─ check the run path below
    │
2.  ├── /readyz returns 503 ──────   read the `metadata.checks` payload
    │                               ─ use the matrix below to find the
    │                                 failing dependency, then jump to
    │                                 the matching troubleshooting § n
    │
    └── /readyz times out ─────────   same path as /healthz timeout above
```

`/healthz` and `/readyz` are unauthenticated by design — you can probe them from anywhere you can reach the service port.

```
 curl -sf http://<host>:<port>/healthz | jq
curl -sf http://<host>:<port>/readyz  | jq '.metadata.checks'
```

`/readyz` failure matrix

`/readyz` runs eight checks; when any fails, the response is `503` with `{"error": "not_ready", "metadata": {"checks": {...}}}`. The keys you will see and what each means:
(kneo_serv/service/routes_health.py)

Check key	What it probes	Common failure	Where to go
`api`	API wiring sentinel	Never fails on its own	If it does, the manager isn't configured — § 1.3
`run_state_store`	`manager.run_state_store.list_runs(limit=1)` succeeds	DB unreachable, schema missing, credentials wrong	§ 2.1 / § 2.2
`continuation_store`	`manager.continuation_store.list()` succeeds	File path missing, permissions wrong, DB issue	§ 2
`queue`	`list_queued_runs(status=…)` returns for `queued` / `running` / `failed`	Queue table missing or DB stall	§ 2, § 5.1
`runtime_registry`	Number of registered runtimes (declared via factories)	Empty registry — service started without runtimes	`extending.md` for runtime registration
`tool_registry`	Number of registered tools	Empty registry — service started without tools	`extending.md`
`providers`	Secrets named in `KNEO_SERV_HEALTH_PROVIDERS` resolve	Provider env var missing or empty	§ 3.2
`mcp`	Secrets named in `KNEO_SERV_HEALTH_MCP_SECRETS` resolve	MCP secret missing	§ 3.2

The payload includes per-check `error` (exception class) and `message` for failed checks, so you usually don't have to guess which subsystem is at fault — copy the message into the relevant troubleshooting section.

Common production incidents

If `/healthz` and `/readyz` are both green but the service is "wrong":

Symptom	First check	Deep dive
All requests return `401`	`Authorization` / `X-Kneo-Api-Key` header is present and valid	§ 4.1, § 4.3
Specific consumer returns `403`	The key's role/scope covers the route	§ 4.2, `security_hardening.md` § 2
Async runs stuck in `queued`	Worker process is up; queue table reachable	§ 5.1
Runs hang mid-workflow	The step's tool or provider call is timing out	§ 5.3
`409 idempotency_key_conflict`	Caller is reusing a key with a different payload	§ 5.4
Tool reports `MissingSecretError`	Provider secret env var is set on the service host	§ 3.1
Logs missing `request_id`	You are reading the right logger (`kneo_serv.service`), not raw uvicorn	§ 7.1, `observability.md`
OpenTelemetry exporter silent	`KNEO_SERV_OTEL_ENABLED=true` and `[telemetry]` extra installed	§ 7.2, `observability.md`
Human task `409 resource_locked`	Another resume is in flight for the same continuation	§ 8.1
Restored backup but state looks stale or mismatched	Stop, re-verify the dump source, follow the recovery shape	§ 2.5, `backup_and_recovery.md`

What to capture before escalating

When the runbook doesn't have an entry that fits, capture this context before paging the on-call developer. It is the same set `troubleshooting.md` asks for in a bug report:

- Service version and commit (`pip show kneo-serv` or the image tag).
- Environment context (`uname -a`, Python version, Postgres version).
- The `request_id` and `run_id` of an affected request.

- `GET /readyz` body, even when it returns `200`.
- For run-shaped problems: `GET /v1/runs/{run_id}`, `GET /v1/runs/{run_id}/trace`, and `GET /v1/runs/{run_id}/checkpoints` (redacted output is fine to share).
- For spec-shaped problems: the output of `kneo spec validate <path> --json`.
- Recent audit events that mention the affected resource: `GET /v1/audit-events?run_id=<id>`.

When to roll back

If the incident started immediately after a deploy and isn't covered by the matrix above:

1. Confirm the deploy is the trigger — diff the running version against the previous version, check the time correlation with the first 503/5xx.
2. If yes, follow `backup_and_recovery.md § Rolling back after a failed upgrade`.

Rolling back when the trigger isn't the deploy throws away forward progress. Diagnose first.

What this page does not cover

- **Severity definitions and paging policy.** Owned by your on-call rota, not by `kneo-serv`.
- **Post-incident review template.** Out of scope.
- **Failure modes during a `kneo-serv` release itself** — those are in `release_checklist.md` and `troubleshooting.md § 9`.

Troubleshooting

Source: `docs/user/troubleshooting.md`

An operator-facing runbook indexed by symptom. Each entry lists the symptom, how to confirm the root cause, and the fix; cross-references point at the authoritative configuration doc when one exists.

If you are responding to a live incident and don't yet have a symptom, start at `incident_response.md` — it walks `/healthz` → `/readyz` → the right section here. This page is the symptom-indexed deep dive.

When you're not sure where to start, check `GET /readyz` (§ 1.2) — it exposes the per-dependency status the service uses internally, and most "service is unhealthy" tickets resolve to one of its check entries.

1. Service won't start or won't accept traffic

1.1 `RuntimeError: KNEO service auth is enabled but no API keys are configured`

The service refuses to start when auth is enabled without keys. `service/auth.py`

- Confirm: check the startup log for the message above.
- Fix: set `KNEO_SERV_API_KEYS` (entries are `name:key:role_or_scope[,role_or_scope]`, semicolon-separated) and/or `KNEO_SERV_ADMIN_API_KEY`. To run without auth, set `KNEO_SERV_AUTH_ENABLED=false` (only for local dev).
- Reference: environment.md § Service Auth, production_readiness_review.md § Role Boundary Review.

1.2 `/readyz` returns 503

`GET /readyz` returns `{"error": "not_ready", "metadata": {"checks": {...}}}` when any dependency check fails. `service/routes_health.py`

- Confirm: `curl -sf http://<host>:<port>/readyz | jq`. Each per-dependency entry has `ok: false` plus `error` and `message` for failed checks.
- Fix: the per-check failure matrix (which check maps to which recovery action) lives in `incident_response.md § /readyz failure matrix`. In summary: store failures are covered by §2; provider/MCP secret failures by §3.

1.3 `RuntimeError: PlatformManager has not been configured`

The default `app` factory configures the platform manager automatically. This error appears when you pass `configure_default_manager=False` to `create_app()` and never call `set_platform_manager()` before serving. `service/dependencies.py`

- Fix: either drop the override, or call `kneo_serv.service.dependencies.set_platform_manager(...)` before the first request.

1.4 `RuntimeError: Invalid KNEO_SERV_API_KEYS entry`

The format is `name:key:role_or_scope[,role_or_scope]` per entry, separated by semicolons. Whitespace inside entries is trimmed; missing colons trigger this error. `service/auth.py`

- Fix: re-render `KNEO_SERV_API_KEYS`. Examples:
- `operator:OP_TOKEN:operator;reviewer:REV_TOKEN:reviewer`
- `svc:SVC_TOKEN:runs:write,human:read,human:write`
- Reference: environment.md § Service Auth, production_readiness_review.md § Route Scope Matrix.

2. Persistence and store failures

2.1 PostgreSQL DSN configured but service falls back to SQLite

The service uses PostgreSQL only when `KNEO_SERV_DATABASE_URL` is set **and** the `[postgres]` or `[deploy]` extra is installed. See `service/factory.py` .

- Confirm: in a dev shell, `python -c "import psycopg; print(psycopg.__version__)"` .
- Fix: install `kneo-serv[deploy]` (Docker image already does this), or `kneo-serv[postgres]` if you don't need telemetry.

2.2 `psycopg.OperationalError` on startup or first request

The DSN can't connect. Common causes: wrong host, missing TLS, wrong credentials, database not yet created.

- Confirm: `psql "$KNEO_SERV_DATABASE_URL" -c '\dt'` from the same network context as the service.
- Fix: correct the DSN, ensure the database exists, and confirm the user has privileges. `KNEO_SERV_DATABASE_URL` must be a libpq-style URL.

2.3 SQLite `database is locked` errors

Concurrent writes to a single SQLite file can collide. The default service worker is single-threaded per process; this typically appears when running multiple service processes against the same SQLite file.

- Fix: switch to PostgreSQL (set `KNEO_SERV_DATABASE_URL`). Multi-process SQLite is not a supported deployment topology.

2.4 Schema migration appears to have run but old data is missing

Migrations are idempotent and version-tracked per store; they don't drop data. If rows look missing after an upgrade, check whether you actually upgraded the same database the service is reading.

- Confirm: compare `KNEO_SERV_DATABASE_URL` (or SQLite path) between the upgrade context and the running service. A stale state file at `.kneo/kneo_runs.sqlite` is a common cause.

2.5 Backup/restore mismatch

`kneo_serv.maintenance.backup_sqlite_database()` produces a file copy that restore expects to find on the same SQLite version line. Restoring across incompatible SQLite versions can fail.

- Fix: align `sqlite3` versions, or migrate to PostgreSQL where backup goes through `pg_dump` / `pg_restore`. See staging_release_runbook.md and release_checklist.md for the seeded recovery drill.

3. Secrets, credentials, and provider integration

3.1 `MissingSecretError` on agent run

Provider keys, MCP credentials, and runtime settings are resolved through env-var references in project config; raw values are never stored. `security/secrets.py`

- Confirm: `kneo config secrets --json` lists which references exist and whether each resolves. The endpoint `GET /security/credentials` exposes the same view (requires `credentials:read`).
- Fix: export the env var named in the error. Set `KNEO_SERV_REQUIRE_PROVIDER_SECRETS=true` to fail fast at startup instead of at first run.

3.2 `GET /readyz` reports missing provider/MCP secrets

Readiness reports the secrets named in `KNEO_SERV_HEALTH_PROVIDERS` and `KNEO_SERV_HEALTH_MCP_SECRETS`. These are operator-curated allowlists, so expect 503 if you list a secret that isn't actually exported.

- Fix: trim the list to secrets you actually use, or export the missing one.

3.3 `kneo spec bundle verify` fails

Bundle verification requires `KNEO_SERV_SPEC_SIGNING_KEY` to match the key used to sign. Bundles signed with a different key (or unsigned) fail verification.

- Fix: rotate the signing key consistently across signing and verifying hosts. The key is HMAC-only; do not commit it.

4. Authentication and authorization

4.1 `401 Unauthorized — A valid Kneo service API key is required`

The route requires auth and the request didn't carry a valid token. `service/auth.py`

- Confirm: send `Authorization: Bearer <key>` or `X-Kneo-Api-Key: <key>`.
- Fix: use one of the configured keys. The CLI service client reads `KNEO_SERV_API_KEY`. For multi-environment workflows use CLI profiles (`kneo config profile use ...`).

4.2 `403 Forbidden — Missing required scope: <scope>`

The token authenticated but the principal does not hold the scope the route requires. `service/auth.py`

- Confirm: the scope in the error body tells you exactly what is missing.
- Fix: assign the principal a role that includes the scope, or add the scope explicitly in `KNEO_SERV_API_KEYS`. See the route ↔ scope matrix in production_readiness_review.md.
- Common gotchas:
- `POST /specs/run` requires `runs:write`, not `specs:read`.
- Reviewer cannot create runs or change policies.
- Service role cannot mutate environment policies (only operator/admin).

4.3 Health endpoints work, all other routes 401

`/healthz`, `/livez`, and `/readyz` are intentionally unauthenticated for load-balancer probes. Everything else is gated by the auth dependency. This is by design; see production_readiness_review.md § Route Scope Matrix.

5. Run lifecycle problems

5.1 Async runs sit in `queued` and never progress

The platform worker is started by `create_default_platform_manager()` in `service/factory.py`. If a custom embedding skips `manager.start_worker()`, queued runs never drain.

- Confirm: `GET /runs?status=queued` shows queued items; `GET /readyz` shows the queue dependency as ok; the service log has no "worker" lines.
- Fix: ensure `start_worker()` is called in the host process after constructing `PlatformManager` directly, or use the default factory.

5.2 Cancelled run still finishes as `succeeded`

Cancellation is cooperative through `CancellationToken` and propagates only at unit-of-work boundaries. A step that completes between the cancel request and the next checkpoint will record its result, but `RunState` remains `cancelled` — the platform does not overwrite cancelled status with completed results.

- Confirm: `GET /runs/{run_id}` should still report `status: cancelled` even if the last checkpoint shows completion of a step.
- If `status` shows `succeeded` after a cancel, file an issue with the run id, the checkpoint timeline (`/runs/{run_id}/checkpoints`), and the trace (`/runs/{run_id}/trace`).

5.3 Run hangs at a workflow step

Workflow steps support `on_error: retry` , `max_retries` , and `timeout_seconds` . If a step has no timeout and the underlying provider/MCP call blocks, the step blocks too.

- Fix: set step-level timeouts, or set the global defaults `KNEO_SERV_PROVIDER_TIMEOUT_SECONDS` / `KNEO_SERV_MCP_TIMEOUT_SECONDS` .

5.4 `409 Conflict — idempotency_key_conflict`

`Idempotency-Key` was reused with a different request body for the same scope. `service/idempotency.py`

- Fix: pick a new key for the new request body, or reuse the same body for the original key. Idempotency records hash the canonical JSON of the request payload and replay the original response on match.

5.5 `400 Bad Request — invalid_idempotency_key`

`Idempotency-Key` headers must be 256 characters or fewer after trimming. `service/idempotency.py`

- Fix: shorten the key. UUIDs are sufficient.

6. Spec validation and compilation

6.1 `SpecCompilationError` with diagnostics

`SpecCompiler` raises this on either schema or semantic validation failure. `spec/compiler.py`

- Confirm: `kneo spec validate <path>` prints the same diagnostics with location info.
- Fix: address each diagnostic. Common causes:
- Missing/extra fields in `version: v1` shape.
- References to undefined components/tools.
- Memory/guardrail policy mis-shape.
- Migrate older specs with `kneo spec migrate <path> --output <new>`.

6.2 `ValueError: Tool '<name>' has no implementation`

The spec references a tool name that is not registered with the `ToolRegistry`. `spec/builder.py`

- Fix: register the tool (programmatically or via MCP import), or remove the reference. The default service registers example tools; toggle with `include_example_tools` when constructing `PlatformManager` directly.

6.3 Inline spec rejected with size error

Inline specs and overrides are bounded. `service/limits.py`

- Fix: tune the relevant limit (`KNEO_SERV_MAX_INLINE_SPEC_BYTES`, `KNEO_SERV_MAX_OVERRIDES_BYTES`, `KNEO_SERV_MAX_METADATA_BYTES`, `KNEO_SERV_MAX_BODY_BYTES`) or move the spec to a path on disk. Limits exist to keep the service from deserializing arbitrarily large payloads.

7. Observability

7.1 Structured logs missing `request_id`

The structured logging middleware always populates `request_id`; missing fields usually mean the log was emitted before `RequestLoggingMiddleware` attached, or you're reading raw uvicorn access logs instead of the service logger.

- Fix: filter by logger name `kneo_serv` or by JSON log format. Clients can supply `X-Request-ID` to override the generated id; the service echoes it on the response.

7.2 OpenTelemetry not exporting

The SDK `OpenTelemetryMiddleware` only attaches when both `KNEO_SERV_OTEL_ENABLED=true` and the `[telemetry]` extra is installed.

- Fix: install `kneo-serv[deploy]` (Docker image) or `kneo-serv[telemetry]`, set `KNEO_SERV_OTEL_ENABLED=true`, and ensure standard OTel exporter env vars (`OTEL_EXPORTER_OTLP_ENDPOINT`, etc.) are set.
- Tool arguments and results are **not** captured by default. Enable with `KNEO_SERV_OTEL_RECORD_ARGUMENTS=true` and/or `KNEO_SERV_OTEL_RECORD_RESULTS=true` only after you've confirmed the data classification allows payload capture.

7.3 Trace events missing for a run

Service-side trace events live in run metadata and at `/runs/{run_id}/trace`. They are emitted by `TracingMiddleware` and the in-process `Tracer`, independent of OTel. Missing events most often mean the run was never executed (e.g. queued and abandoned) or the spec disabled tracing.

- Fix: confirm the run reached `running` / `succeeded`. If the workflow middleware list omits `TracingMiddleware`, restore it (the default chain includes it).

8. Human-in-the-loop

8.1 `LockAcquisitionError` on resume

A `POST /human-tasks/{continuation_id}/resume` failed because another caller currently holds the resume lock for the same continuation. `platform/manager.py`

- Confirm: the error body identifies the lock name. The first caller is still in flight.
- Fix: wait for the in-flight resume to complete; do not retry blindly. Use idempotency keys on resume to make retries safe.

8.2 Continuation expired or missing

If the continuation store was rotated (e.g. `.kneo/continuations` recreated, or PostgreSQL row deleted), `/human-tasks/{continuation_id}` returns 404.

- Fix: the run cannot be resumed. Start a new run.

9. Release and supply chain

For release-flow issues (mypy, pip-audit, build, tag, publish), follow release_checklist.md and supply_chain_review.md. The release workflow at `.github/workflows/release.yml` emits the gate

that failed in its job summary.

What to capture before opening a bug

When a problem isn't covered above:

- Service version and commit (`pip show kneo-serv`, plus the git commit if installed from source).
- Environment context (`uname -a`, Python version, Postgres version if used).
- The `request_id` and `run_id` from logs.
- `GET /readyz` body.
- For run problems: `GET /runs/{run_id}`, `GET /runs/{run_id}/trace`, and `GET /runs/{run_id}/checkpoints` (redacted output is fine to share).
- For spec problems: the output of `kneo spec validate <path> --json`.

Audit events are accessible at `GET /audit-events` and frequently contain the operator action that preceded a fault.

0.11.0 behavior notes

0.11.0 is a breaking cut; see upgrade.md § 0.11.0 for the full migration. The behaviors most likely to surface as a "bug" after upgrading:

Async run-create now returns `202 Accepted` (was `200`)

`POST /runs` / `POST /v1/runs` (and `POST /specs/run`) with `async_mode=true` now return **`202`**, not `200`. Synchronous creates still return `200`, the response body is unchanged, and idempotency-replay preserves the original `202`. A client asserting `== 200` on an async create will read this as a failure — accept `2xx` (or `202` specifically). Poll `GET /runs/{run_id}` as before.

`422 unknown_query_parameters` on a request that used to work

A query parameter the route does not declare is now **rejected** (it was silently ignored through 0.10.x). The response detail names the offending keys. This is almost always a typo (`?staus=failed`) or a client sending a param the endpoint never supported — fix the parameter name. Monitoring/scrape routes (`/healthz`, `/livez`, `/readyz`, `/metrics`) are exempt and still accept arbitrary params.

A run now aborts on a guardrail that previously did nothing

In 0.11.0 a **tool-stage guardrail with a raising action** (`block`, escalate, human_review, retry, revise) aborts the run — sync → `422 guardrail_violation`, async → `failed` — and **`workflow`-stage guardrails are enforced per step**. Before 0.11.0 a tool-stage `block` failed open (the run continued)

and a workflow-stage guardrail was refused at `/specs/validate`. If a run that used to complete now stops, a previously-inert guardrail is now doing its job — this is the intended fail-closed behavior. (`redact` / `warn` rewrite/log instead of aborting; non-`middleware` modes are still rejected at validate.)

0.9.0 behavior notes

Tools echo their arguments back instead of running

A pre-0.9.0 bug: spec-compiled bridge agents got no tool handlers at all, so every declared tool silently returned its own arguments. Upgrade — 0.9.0 wires handler dispatch (and enforces the spec `ToolPolicy` on it).

A stdio/sse MCP tool fails or stalls on every call

Pre-0.9.0, MCP sessions were created on a throwaway event loop — stdio/sse transports failed on every invocation. 0.9.0 hosts each session on a dedicated loop; a dead server now aborts the connect after `KNEO_SERV_MCP_CONNECT_TIMEOUT_SECONDS` instead of hanging the tool.

`GET /runs/{id}/trace` is empty for a failed or resumed run

Fixed in 0.9.0: failed runs persist their collected trace, and the pre-pause timeline survives a human-task resume. Older runs that predate the fix reassemble from their checkpoints where events exist.

Resume or `/continue` returns `409 run_state_conflict`

Expected: the run's status forbids the operation (e.g. it was cancelled after pausing, or a live worker still holds its lease). The response carries the run's current status — re-fetch the run rather than retrying blindly.

A run fails with `422 guardrail_violation`

A content guardrail blocked the input/output/tool result — previously this surfaced as an opaque `500 internal_error`. The violation type is in the response detail.

Deployment smoke test

Source: `docs/user/deployment_smoke.md`

This smoke test validates a running service deployment through the public HTTP API. It uses the self-contained `examples/smoke_human_workflow.yaml` spec and covers health, readiness, auth behavior, spec validation, run creation, human resume, audit listing, credential inventory, and environment policy updates.

Compose stack

Prepare a production env file:

```
cp deploy/production.env.example deploy/production.env
```

`deploy/production.env` is intentionally ignored by source control. Replace all placeholder tokens and passwords before binding a deployment to a real network. For local CI/smoke runs, the documented placeholder values are used only to exercise the auth path.

Start the API and PostgreSQL:

```
docker compose --env-file deploy/production.env up --build -d
```

Run the smoke test against the unversioned routes:

```
python scripts/deployment_smoke.py \
  --base-url http://127.0.0.1:8000 \
  --api-key replace-admin-token \
  --operator-api-key replace-operator-token \
  --reviewer-api-key replace-reviewer-token \
  --viewer-api-key replace-viewer-token \
  --expect-auth
```

Run the same smoke test against the versioned routes:

```
python scripts/deployment_smoke.py \
  --base-url http://127.0.0.1:8000 \
  --api-prefix /v1 \
  --api-key replace-admin-token \
  --operator-api-key replace-operator-token \
  --reviewer-api-key replace-reviewer-token \
```

```
  --viewer-api-key replace-viewer-token \
  --expect-auth
```

Shut the stack down when finished:

```
docker compose --env-file deploy/production.env down
```

PostgreSQL coverage

The compose stack uses PostgreSQL by default through `KNEO_SERV_DATABASE_URL=postgresql://...@db:5432/...`, so the smoke path above also validates PostgreSQL-backed run state, checkpoints, continuations, queue records, locks, audit events, and project metadata.

For a separately managed PostgreSQL database, start the API with `KNEO_SERV_DATABASE_URL` set and run the same `scripts/deployment_smoke.py` commands against that service URL.

Staging and remote smoke

Prepare a staging env file from the example:

```
cp deploy/staging.env.example deploy/staging.env
```

Replace every placeholder token, database password, provider key, and telemetry endpoint before use. For compose-based staging rehearsals, set `KNEO_SERV_ENV_FILE` so the API container reads the staging file:

```
python scripts/validate_staging_env.py deploy/staging.env
```

The validator fails if required staging settings are missing, scoped API roles are incomplete, payload telemetry capture is enabled, provider secret checks are disabled, or placeholder values remain.

For repeatable staging gates, render the local file from secret-backed environment variables instead of editing it by hand:

```
 export KNEO_STAGING_API_KEYS="operator:<operator-token>:operator;reviewer:<reviewer-
token>:reviewer;viewer:<viewer-token>:viewer"
export KNEO_STAGING_ADMIN_API_KEY=<admin-token>
export KNEO_STAGING_SPEC_SIGNING_KEY=<signing-key>
export KNEO_STAGING_DATABASE_URL=<postgresql-dsn>
export KNEO_STAGING_POSTGRES_PASSWORD=<compose-db-password>
export KNEO_STAGING_OTEL_EXPORTER_OTLP_ENDPOINT=<otel-endpoint>
```

```
export KNEO_STAGING_OPENAI_API_KEY=<openai-key>
export KNEO_STAGING_MCP_API_KEY=<mcp-key>
python scripts/render_staging_env.py --output deploy/staging.env
```

```
KNEO_SERV_ENV_FILE=./deploy/staging.env \
  docker compose --env-file deploy/staging.env up --build -d
```

For a remote staging deployment, run the smoke script against the public staging URL with scoped keys:

```
export KNEO_STAGING_BASE_URL=https://staging.example.com
export KNEO_STAGING_OPERATOR_TOKEN=<operator-token>
export KNEO_STAGING_REVIEWER_TOKEN=<reviewer-token>
export KNEO_STAGING_VIEWER_TOKEN=<viewer-token>
python scripts/deployment_smoke.py --api-prefix /v1 --expect-auth
```

The operator key validates specs, creates runs, reads audit and credential inventory, and updates policy state. The reviewer key resumes the human task. The viewer key is optional; when supplied, the smoke verifies that policy writes are rejected with `403`.

The full staging release gate validates `deploy/staging.env`, derives the scoped smoke tokens from `KNEO_SERV_API_KEYS`, and runs the `/v1` remote smoke:

```
python scripts/staging_release_gate.py \
  --env-file deploy/staging.env \
  --base-url https://staging.example.com
```

The GitHub Actions `Staging Gate` workflow runs the same renderer and release gate from the `staging` environment secrets. Dispatch it with the deployed staging URL after the service is reachable.

Self-hosted staging rehearsal (no standing staging)

A release cycle without a standing staging deployment can still run both staging gates against a *real deployed instance*: deploy the published release-candidate image on any Docker host and point the gates at it. This is the procedure that first closed the gate at `v0.9.0rc1` (it had been deferred for four releases for want of a staging environment).

1. **Generate real secret values** (no placeholders — the validator rejects them) and export the eight `KNEO_STAGING_*` variables from the renderer section above. Generate strong random tokens; for a compose-hosted rehearsal, point `KNEO_STAGING_DATABASE_URL` at the compose service host (`@db:5432`).

2. **Store the same values as GitHub environment secrets** so the hosted workflow renders an env that matches the deployment:

```
bash gh api -X PUT repos/<owner>/<repo>/environments/staging --silent for var in KNEO_STAGING_API_KEYS KNEO_STAGING_ADMIN_API_KEY \ KNEO_STAGING_SPEC_SIGNING_KEY KNEO_STAGING_DATABASE_URL \ KNEO_STAGING_POSTGRES_PASSWORD \ KNEO_STAGING_OTEL_EXPORTER_OTLP_ENDPOINT \ KNEO_STAGING_OPENAI_API_KEY KNEO_STAGING_MCP_API_KEY; do printenv "$var" | gh secret set "$var" --env staging done
```

3. **Deploy the rc image itself, pinned by digest** (verify the cosign signature first — see security_hardening.md), under the rendered staging env:

```
bash python scripts/render_staging_env.py --output deploy/staging.env docker pull ghcr.io/<owner>/kneo-serv@sha256:<rc-digest> docker tag ghcr.io/<owner>/kneo-serv@sha256:<rc-digest> \ ghcr.io/<owner>/kneo-serv:latest # local-only alias for compose KNEO_SERV_ENV_FILE=./deploy/staging.env \ docker compose --env-file deploy/staging.env up -d --no-build
```

4. **Run the script gate locally** against the instance (`scripts/staging_release_gate.py --env-file deploy/staging.env --base-url http://127.0.0.1:8000`), then **dispatch the hosted `Staging Gate` workflow** with a URL the GitHub runner can reach.

Reachability caveat: GitHub-hosted runners have **no IPv6 egress** — a direct IPv6 URL fails with `[Errno 101] Network is unreachable` even when the host is genuinely on the IPv6 internet. If the host has no public IPv4, front the instance with a short-lived tunnel and pass the tunnel URL instead:

```
bash cloudflared tunnel --url http://localhost:8000 --no-autoupdate & # grep the https://<random>.trycloudflare.com URL from its output gh workflow run staging-gate.yml -f staging_url=https://<random>.trycloudflare.com
```

Auth stays enforced for the whole exposure window — the gate's `--expect-auth` checks depend on it.

5. **Tear down afterwards**: kill the tunnel, `docker compose --env-file deploy/staging.env down -v`, delete the rendered `deploy/staging.env`, and remove the local `latest` alias. The GitHub environment secrets from step 2 are rehearsal-scoped — re-render them against real keys and a managed DSN if you later stand up permanent staging.

See also

- `backup_and_recovery.md § Data-only restore into a clean volume` — the seeded recovery drill that pairs with the scoped Compose smoke above. Run the smoke first to

populate run, checkpoint, audit, and policy state, then exercise the data-only restore to verify it survives a fresh volume.